A Dream of Spring for Open-Weight LLMs: 10 Architectures from Jan-Feb 2026

A Round Up And Comparison of 10 Open-Weight LLM Releases in Spring 2026

SEBASTIAN RASCHKA, PHD
FEB 25, 2026

If you have struggled a bit to keep up with open-weight model releases this month, this article should catch you up on the main themes.

In this article, I will walk you through the ten main releases in chronological order, with a on the architecture similarities and differences:

1. Arcee AI's Trinity Large (Jan 27, 2026)
2. Moonshot AI's Kimi K2.5 (Jan 27, 2026)
3. StepFun Step 3.5 Flash (Feb 1, 2026)
4. Qwen3-Coder-Next (Feb 3, 2026)
5. z.AI's GLM-5 (Feb 12, 2026)
6. MiniMax M2.5 (Feb 12, 2026)
7. Nanbeige 4.1 3B (Feb 13, 2026)
8. Qwen 3.5 (Feb 15, 2026)
9. Ant Group's Ling 2.5 1T & Ring 2.5 1T (Feb 16, 2026)
10. Cohere's Tiny Aya (Feb 17, 2026)
11. Update 1: Sarvam 30B and 105B (Mar 6, 2026)

(PS: DeepSeek V4 will be added once released.)

Since there's a lot of ground to cover, I will be referencing my previous The Big LLM Architecture Comparison article for certain technical topics (like Mixture-of-Experts, Q

Norm, Multi-head Latent Attention, etc.) throughout this article for background informa
avoid redundancy in this article.

1. Arcee AI's Trinity Large: A New US-Based Start-Up Sharing Open-Weight Models

On January 27, Arcee AI (a company I hadn't had on my radar up to then) began releas
versions of their open-weight 400B Trinity Large LLMs on the model hub, along with tv
smaller variants:

- Their flagship large model is a 400B param Mixture-of-Experts (MoE) with 13B ac
parameters.
- The two smaller variants are Trinity Mini (26B with 3B active parameters) and Trini
Nano (6B with 1B active parameters).

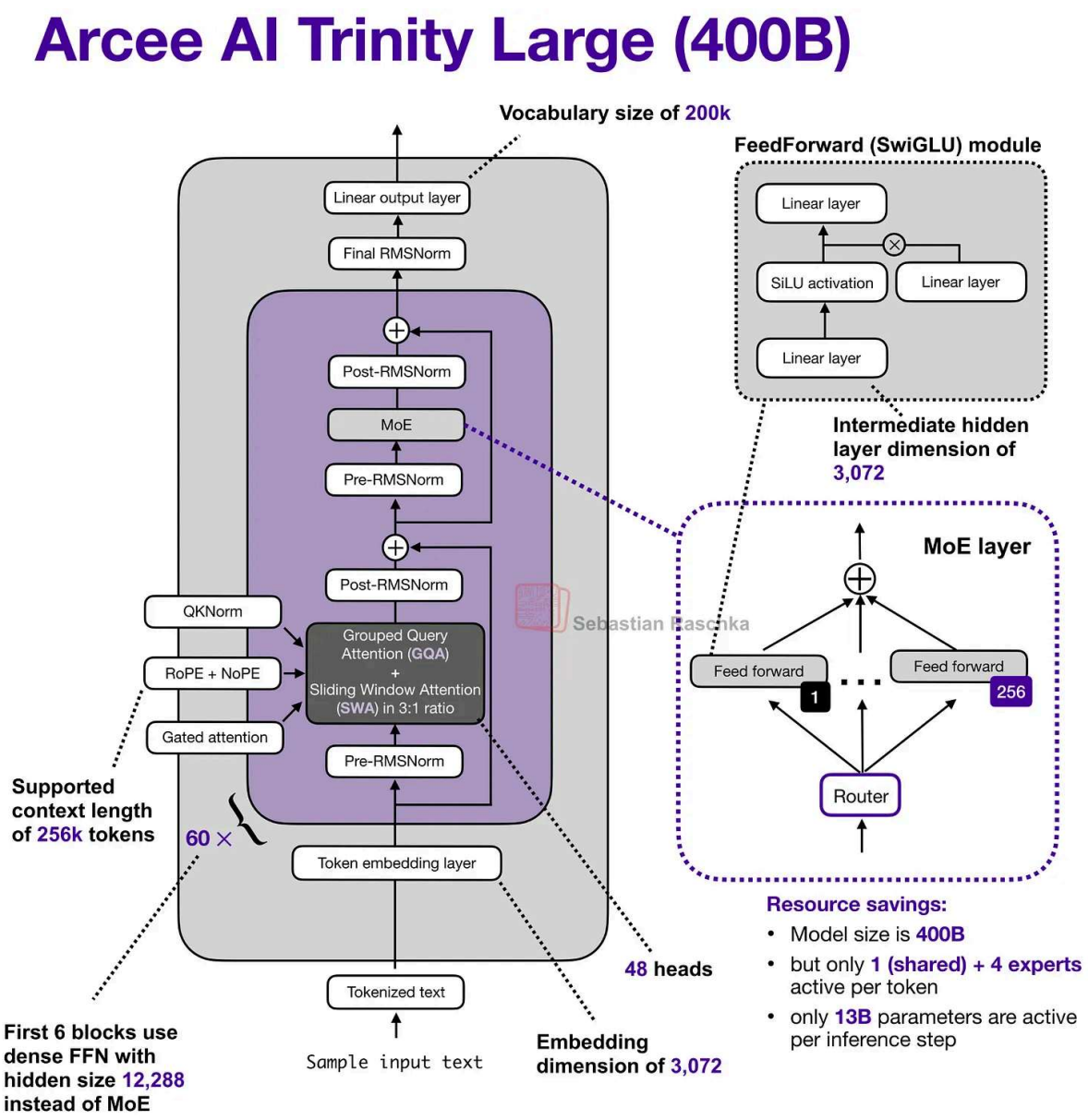


Figure 1: Overview of the Trinity Large architecture (based on the model hub config file).

Along with the model weights, Arcee AI also released a nice technical report on GitHub Feb 18 also on arxiv) with lots of details.

So, let's take a closer look at the 400B flagship model. Figure 2 below compares it to z. GLM-4.5, which is perhaps the most similar model due to its size with 355B paramete

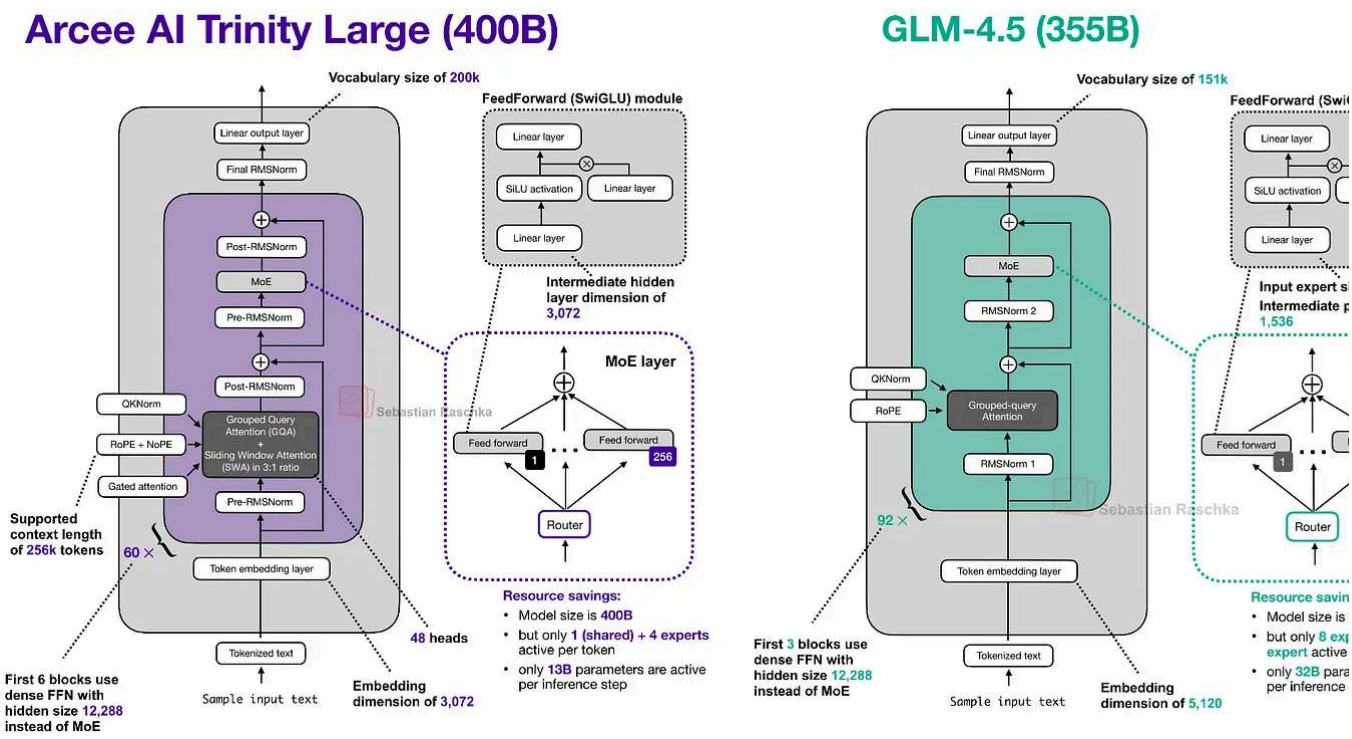


Figure 2: Arcee AI Trinity Large next to GLM-4.5 of a relatively similar size (400B vs 355B).

As we can see in the Trinity and GLM-4.5 comparison, there are several interesting architectural components added to the Trinity model.

First, there are the alternating local:global (sliding window) attention layers (SWA) like i Gemma 3, Olmo 3, Xiaomi MiMo, etc. In short, SWA is a type of sparse (local) attention pattern where each token attends only to a fixed-size window of t recent tokens (for ex 4096) instead of attending to the entire input (which could be up to $n=256,000$ token This reduces the per-layer regular attention cost from $O(n^2)$ to roughly $O(n \cdot t)$ for seque length n, which is why it is attractive for long-context models.

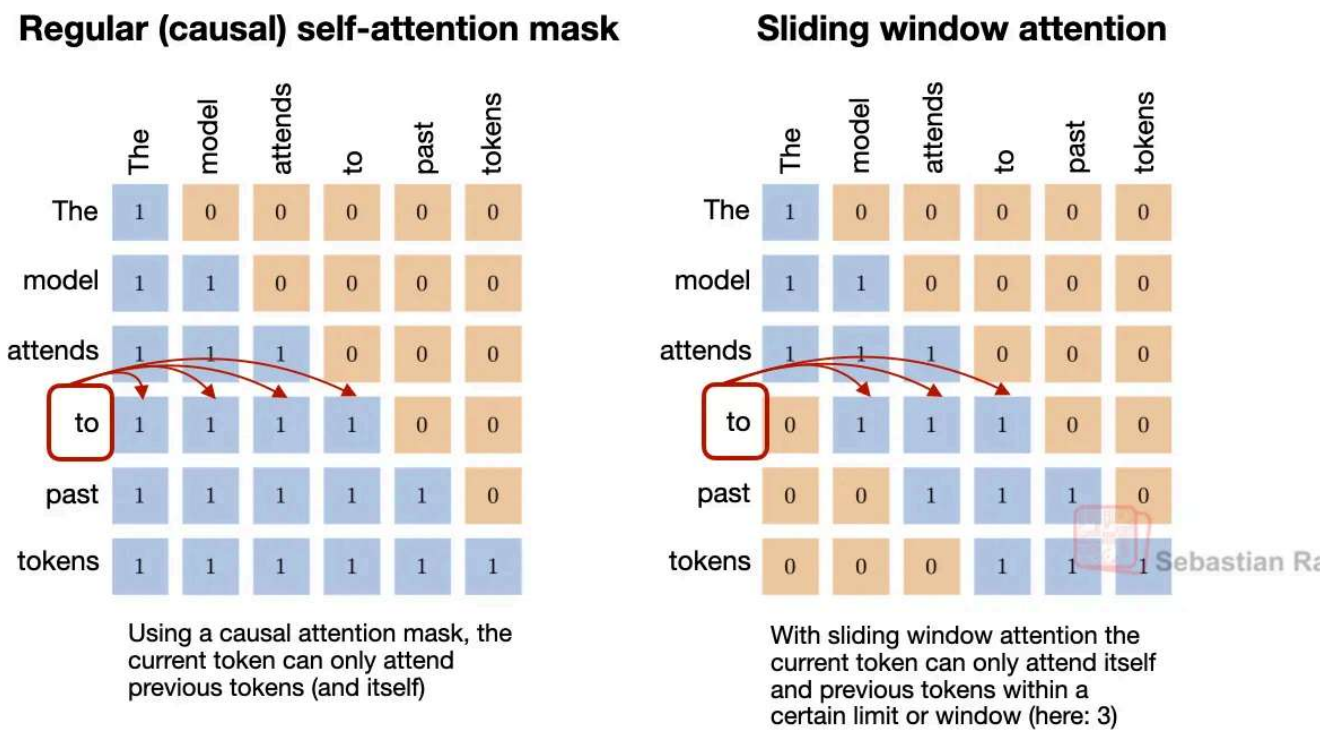


Figure 3: A comparison between regular attention (global attention) and sliding window attention (local attention).

But instead of using the common 5:1 local:global ratio that Gemma 3 and Xiaomi used Arcee team opted for a 3:1 ratio similar to Olmo 3, and a relatively large sliding window 4096 (also similar to Olmo 3).

The architecture also uses QK-Norm, which is a technique that applies RMSNorm to th and queries to stabilize training (as shown in Figure 4 below), as well as no positional embeddings (NoPE) in the global attention layers similar to SmolLM3.

Trinity also has a form of gated attention. It's not a full-blown Gated DeltaNet but it use similar gating as in the attention mechanism in Qwen3-Next.

I.e., the Trinity team modified the standard attention by adding elementwise gating to th scaled dot-product before the output linear projection (as shown in the figure below), w reduces attention sinks and improves long-sequence generalization. Additionally, it als helped with training stability.

```python
def attn_trinity(x, Wq, Wk, Wv, Wo, Wg, *, h):
    B, T, D = x.shape
    dh = D // h
    q = (x @ Wq.T).view(B, T, h, dh).transpose(1, 2)
    k = (x @ Wk.T).view(B, T, h, dh).transpose(1, 2)
    v = (x @ Wv.T).view(B, T, h, dh).transpose(1, 2)

    q = rmsnorm(q)  # QK-Norm
    k = rmsnorm(k)

    o = sdpa(q, k, v) # standard scaled dot product attention

    ### Gating ###
    g = torch.sigmoid(x @ Wg.T).view(B, T, h, dh).transpose(1, 2)
    o = o * g  # Elementwise gate AFTER attention before Wo output projection
    ##############

    return (o.transpose(1, 2).reshape(B, T, D)) @ Wo.T
```

Figure 4: Illustration of the gating mechanism that Trinity Large uses in the attention mechanism.

Also, the Trinity technical report showed that the modeling performance of the Trinity L and GLM-4.5 base models are practically identical (I assume they didn't compare it to recent base models because many companies only share their fine-tuned models thes days.)

You may have noticed the use of four (instead of two) RMSNorm layers in the previous Large architecture figure which looks similar to Gemma 3 at first glance.

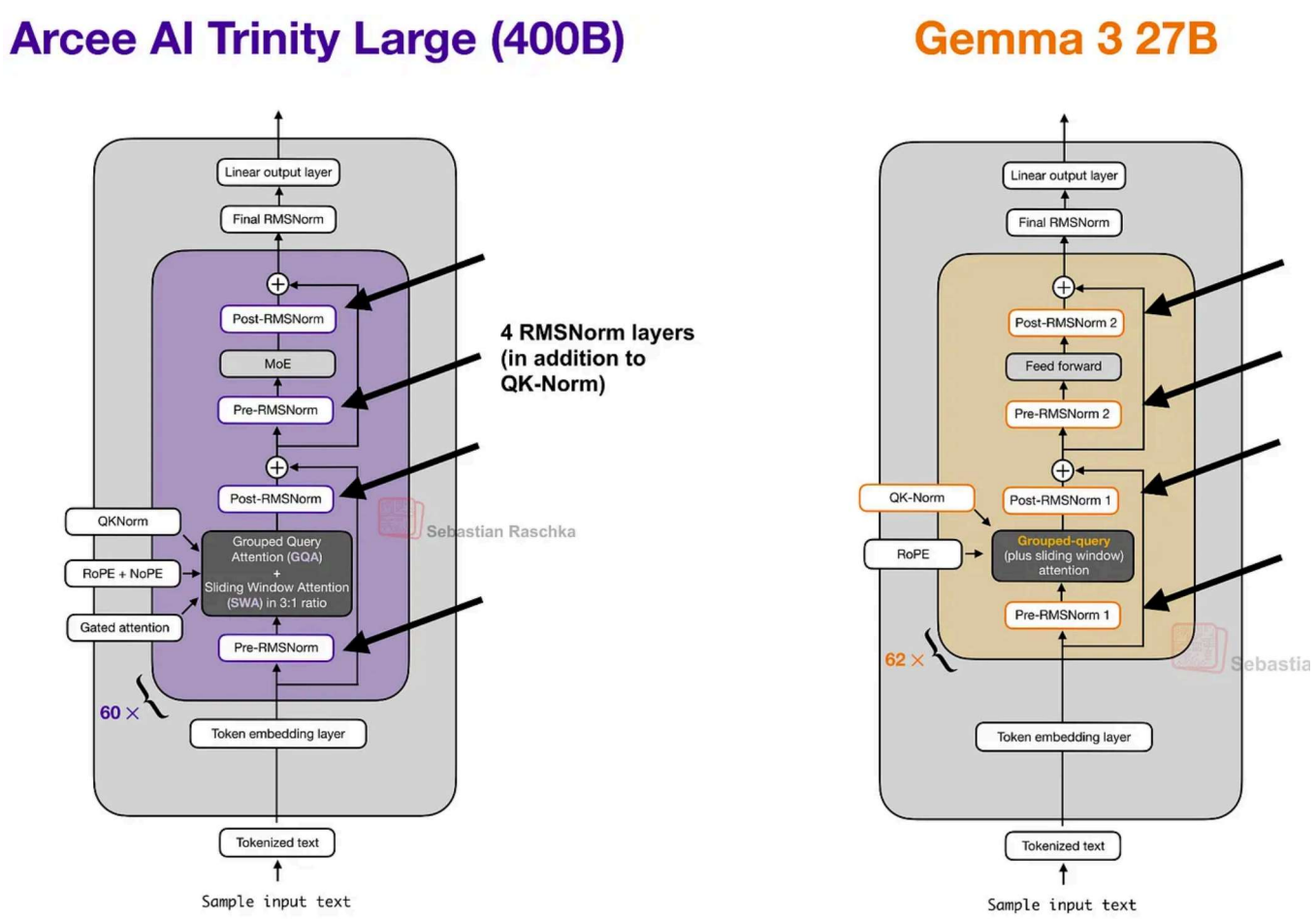


Figure 5: Arcee Trinity and Gemma 3 RMSNorm placement side by side.

Overall, the RMSNorm placement looks like a Gemma 3-like RMSNorm placement, but twist here is that the gain of the second RMSNorm (in each block) is depth-scaled, me it's initialized to about 1 / sqrt(L) (with L the total number of layers). So, early in training, residual update starts small and grows as the model learns the right scale.

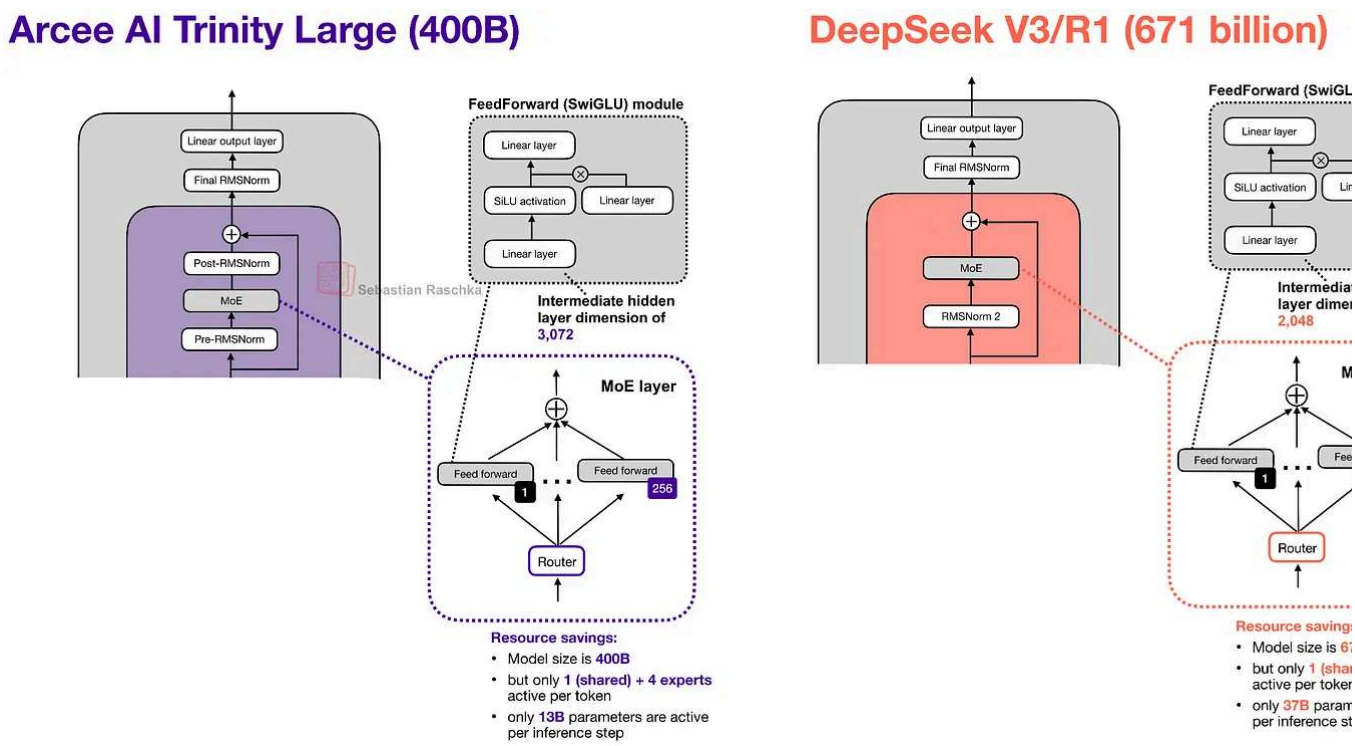


Figure 6: Arcee Trinity and DeepSeek V3/R1 MoE side by side.

The MoE is a DeepSeek-like MoE with lots of small experts, but made it coarser as that with inference throughput (something we have also seen in Mistral 3 Large when they adopted the DeepSeek V3 architecture).

Lastly, there are some interesting details on the training improvements (a new MoE load balancing strategy and another using the MuOpt optimizer), but since this is a mainly a architecture article (and there are many more open-weight LLMs to cover), these detai out of scope.

2. Moonshot AI's Kimi K2.5: A DeepSeek-Like Model at a 1-Trillion-Parameter Sca

While Arcee Trinity essentially matched the modeling performance of the older GLM-4 model, Kimi K2.5 is an open-weight model that set a new open-weight performance ce at the time of its release on Jan 27.

Impressively, according to their own benchmarks in their detailed technical report, it wa par with the leading proprietary models at the time of its release.

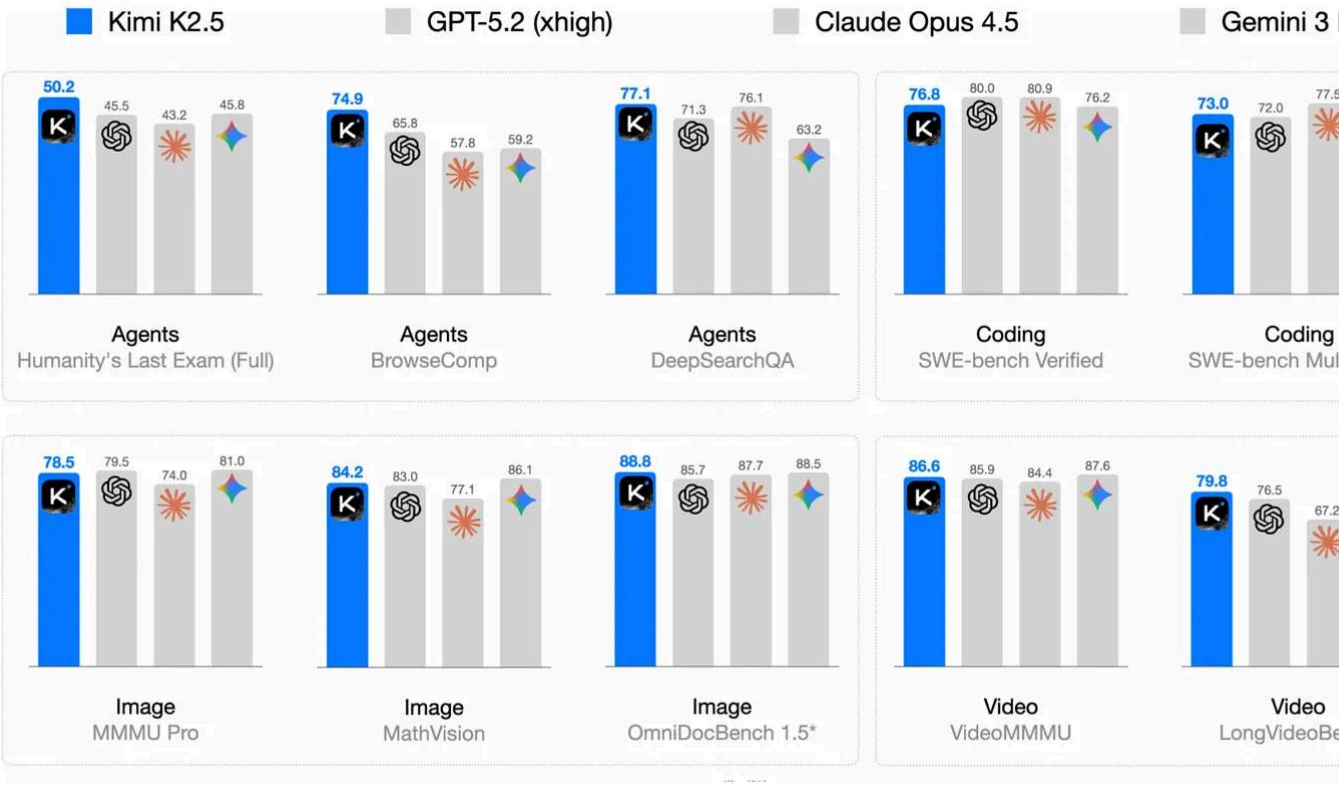


Figure 7: Kimi K2.5 performance benchmark from the official K2.5 technical report.

The good modeling performance is no surprise when compared to, e.g., Arcee Trinity or GLM-4.5 covered earlier, since (similar to its K2 predecessor), Kimi K2.5 is a 1-trillion-parameter model and thus 2.5x larger than Trinity and 2.8x larger than GLM-4.5.

Overall, the Kimi K2.5 architecture is similar to Kimi K2, which, in turn, is a scaled-up ve of the DeepSeek V3 architecture.

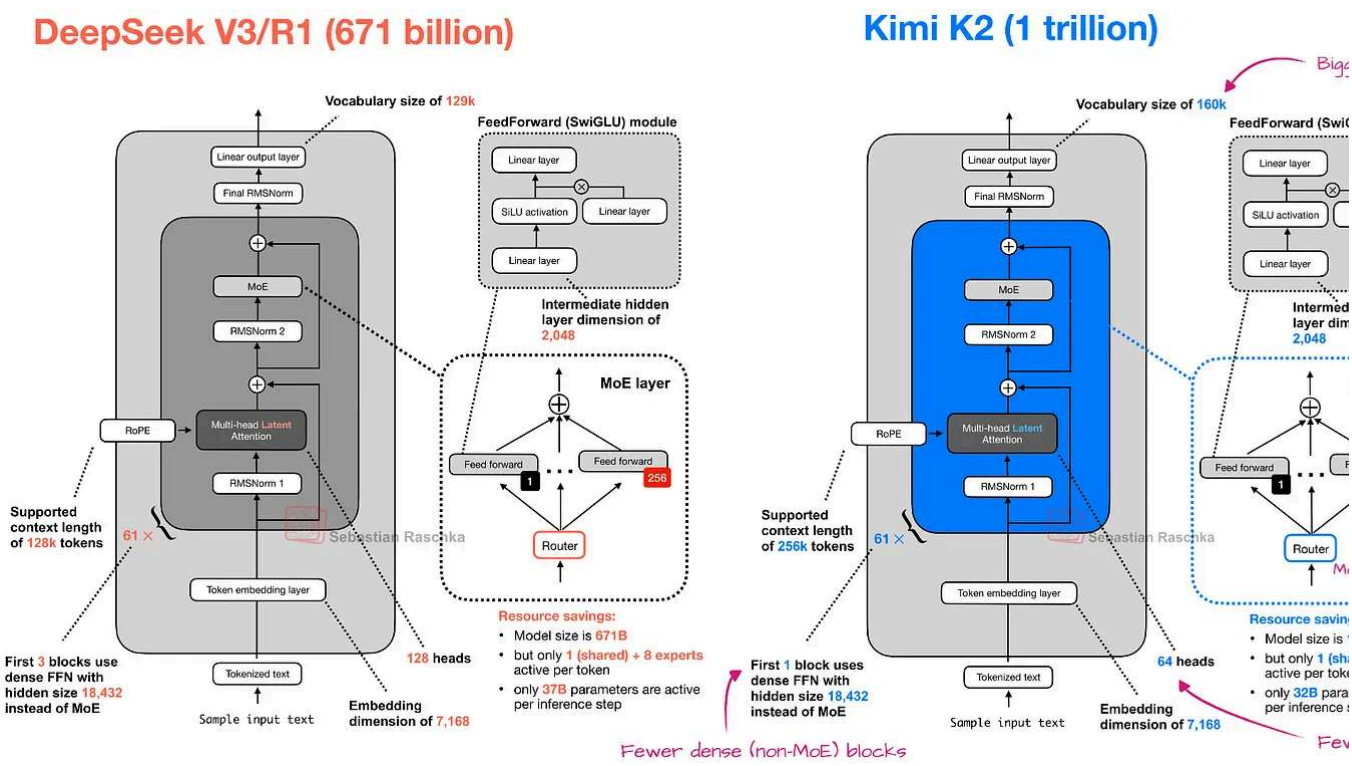


Figure 8: Kimi K2 is a larger version of the DeepSeek V3 architecture.

However, K2 was a pure text model, and Kimi K2.5 is now a multimodal model with visi support. To quote from the technical report:

> *Kimi K2.5 is a native multimodal model built upon Kimi K2 through large-scale joint training on approximately 15 trillion mixed visual and text tokens.*

During the training, they adopted an early fusion approach and passed in the vision tok early on alongside the text tokens, as I discussed in my older Understanding Multimod LLMs article.

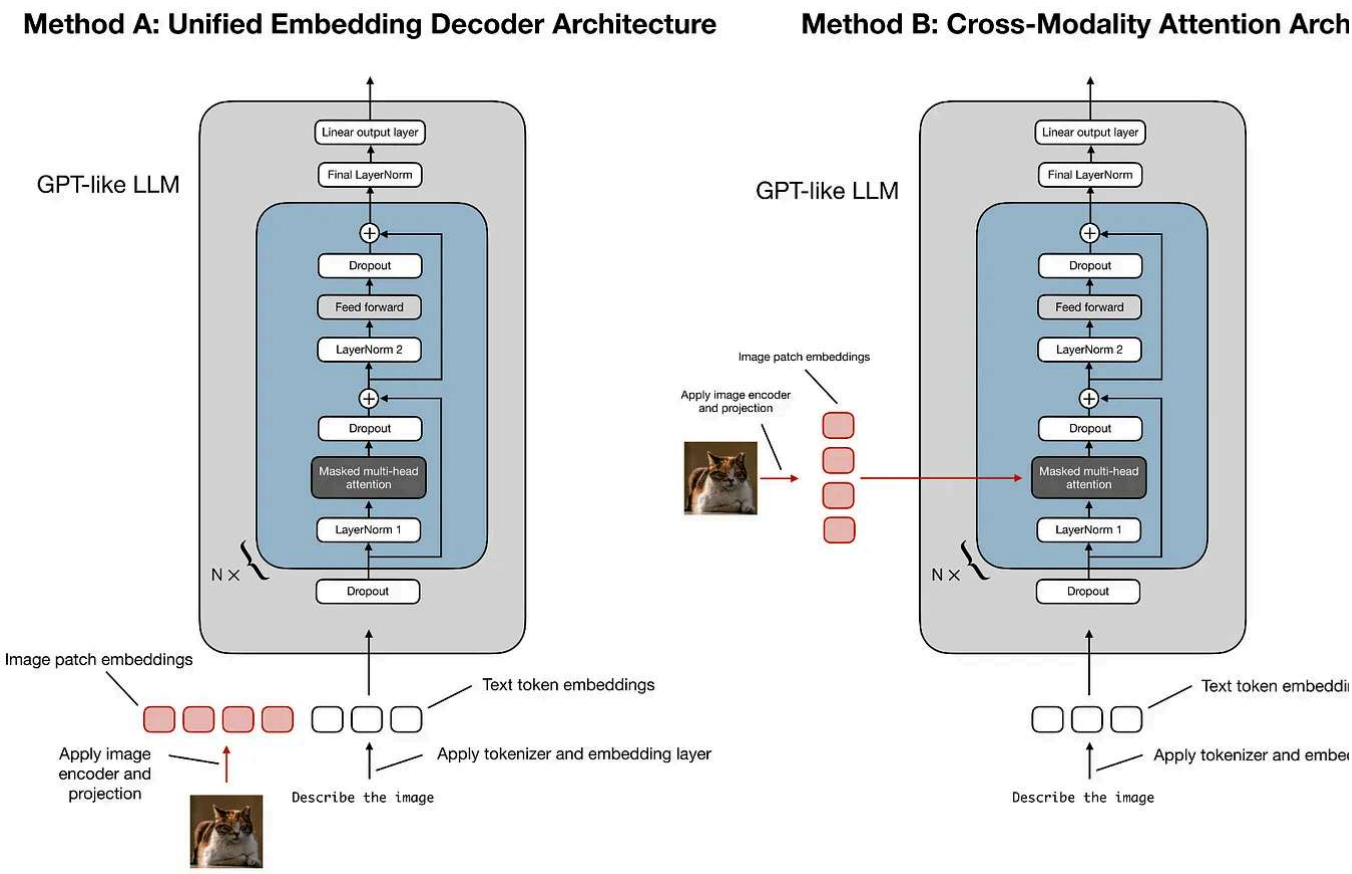


Figure 9: Like most other contemporary multimodal LLMs, Kimi K2.5 uses method A, passing the vision tokens alongside the text tokens during training.

Side note: In multimodal papers, "early fusion" is unfortunately overloaded. It can mean

1. When the model sees vision tokens during pre-training. I.e., vision tokens are mixed i the start (or very early) of pre-training as opposed to later stages.

2. How the image tokens are combined in the model. I.e., they are fed as embedded tok alongside the text tokens.

In this case, while the term "early fusion" in the report specifically refers to point 1 (whe vision tokens are provided during pre-training), point 2 is also true here.

Furthermore, regarding point 1, the researchers included an interesting ablation study showing that the model benefits from seeing vision tokens early in pre-training, as sho the annotated table below.

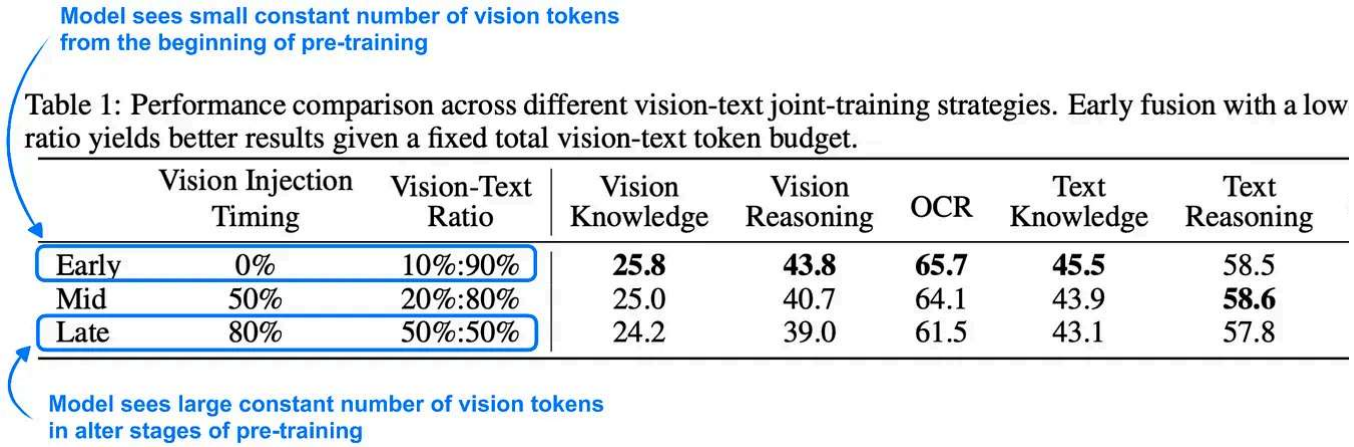


Table 1: Performance comparison across different vision-text joint-training strategies. Early fusion with a low ratio yields better results given a fixed total vision-text token budget.

	Vision Injection Timing	Vision-Text Ratio	Vision Knowledge	Vision Reasoning	OCR	Text Knowledge	Text Reasoning
Early	0%	10%:90%	**25.8**	**43.8**	**65.7**	**45.5**	58.5
Mid	50%	20%:80%	25.0	40.7	64.1	43.9	**58.6**
Late	80%	50%:50%	24.2	39.0	61.5	43.1	57.8

Figure 10: Given a fixed number of vision tokens during training, the model performance benefits if the model is shown a smaller number of vision tokens early on during pre-training (as opposed to adding a higher number of vision tokens later on). Annotated table from the Kimi K2.5 technical report.

3. StepFun's Step 3.5 Flash: Good Performance at Great Tokens/Sec Throughput

I have to admit that I haven't had the Step models on my radar yet. This one caught my attention due to its interesting size, detailed technical report, and fast tokens/sec performance.

Step 3.5 Flash is a 196B parameter model that is more than 3x smaller than the recent DeepSeek V3.2 model (671B) while being slightly ahead in modeling performance benchmarks. According to the Step team, Step 3.5 Flash has a 100 tokens/sec throughput a 128k context length, whereas DeepSeek V3.2 has only a 33 tokens/sec throughput on Hopper GPUs, according to the data on the Step model hub page.

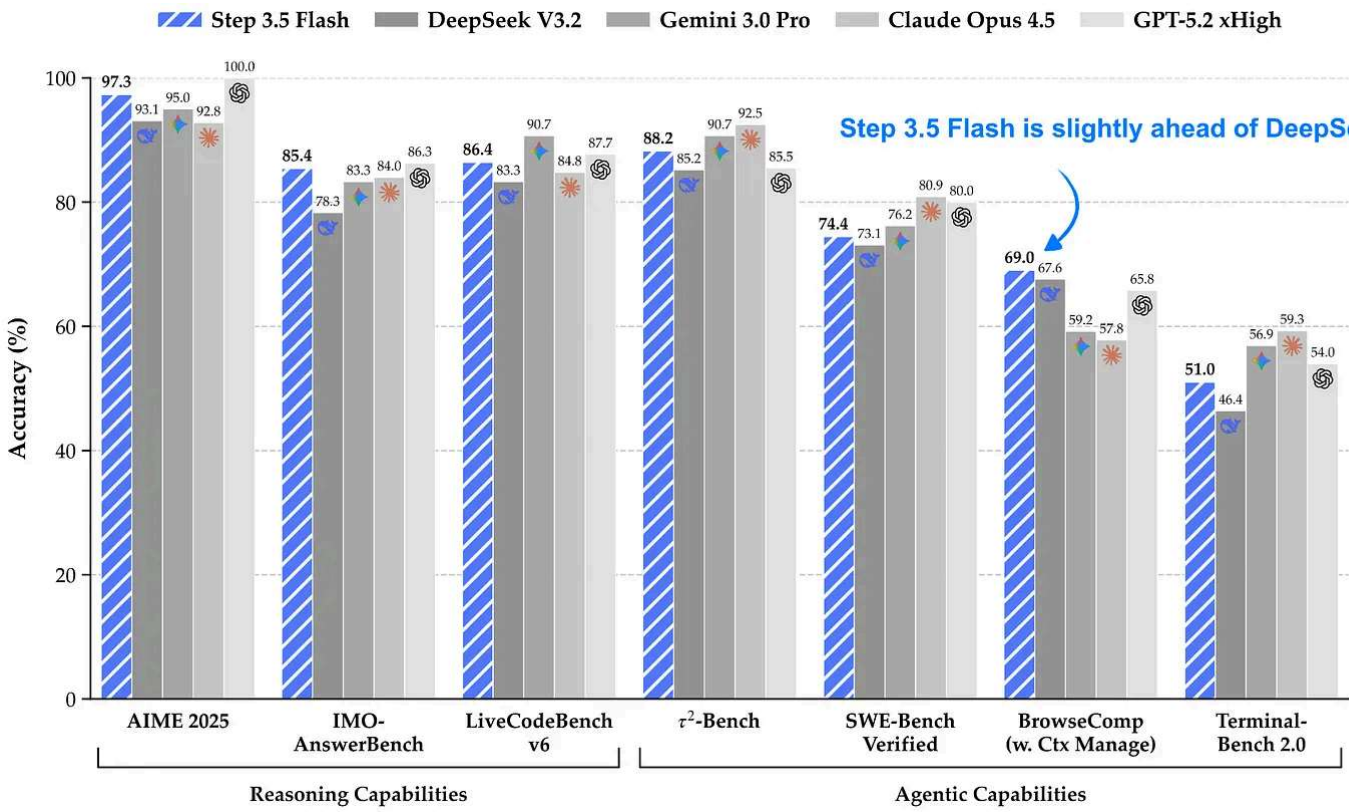


Figure 11: Step 3.5 Flash benchmark from the Step technical report.

One reason for this higher performance is the model's smaller size (196B-parameter M with 11B parameters active per token versus 671B-parameter MoE with 37B paramete active), as shown in the figure below.

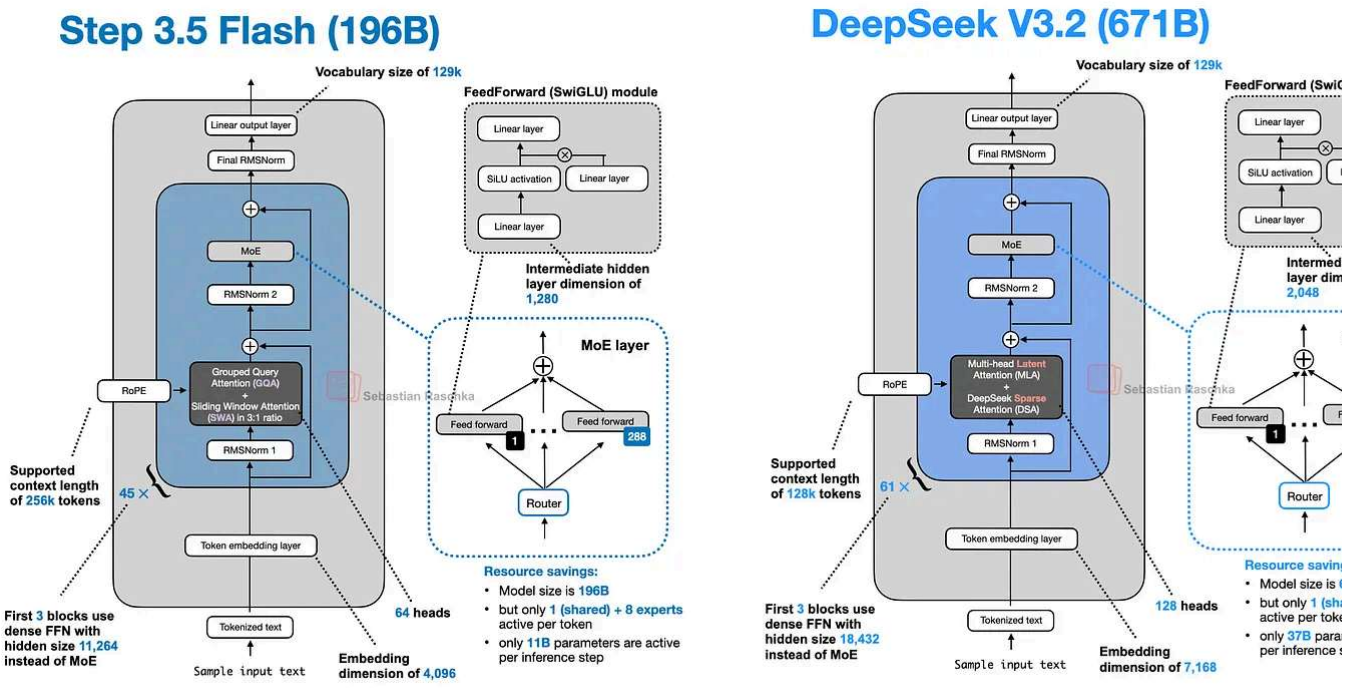


Figure 12: Step 3.5 Flash and DeepSeek V3.2 side by side.

The other reason along with gated attention (which we previously discussed in the cor Trinity) is Multi-Token Prediction (MTP). DeepSeek has been an early adopter of multi- prediction, a technique that trains the LLM to predict multiple future tokens at each ste rather than a single one. Here, at each position t, small extra heads (linear layers) outpu for t+1...t+k, and we sum cross-entropy losses for these offsets (in the MTP paper, the researchers recommended k=4).

This additional signal speeds up training, and inference may remain at generating one t at a time, as illustrated in the figure below.

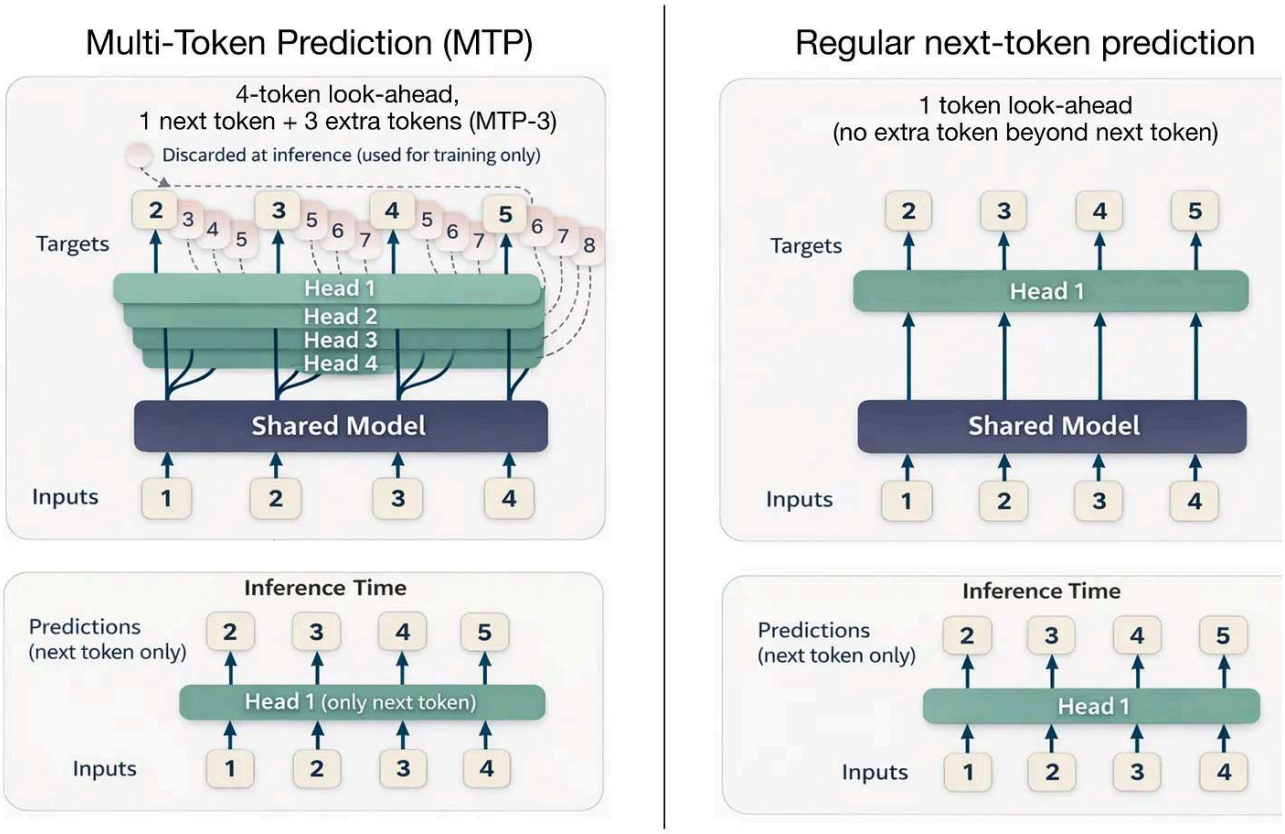


Figure 13: Multi-Token Prediction versus regular next token prediction. (Left subfigure inspired by the MTP paper.) Originally, MTP was only used during training, not inference; hence, the inference time steps (bottom) show a single next-token prediction.

DeepSeek V3 reported using MTP-1, that is, MTP with 1 extra token (instead of 3) duri training, and then making MTP optional during inference.

Step 3.5 Flash uses MTP with 3 additional tokens (MTP-3) during both training and inference (note that MTP is usually not used during inference, and this is an exception)

Note that the previously discussed Arcee Trinity and Kimi K2.5 do not use MTP, but oth architectures already use an MTP-3 setup similar to Step 3.5 Flash, for example, GLM- and MiniMax M2.1.

4. Qwen3-Coder-Next: An Attention-Hybrid for Coding

In early February 2026, the Qwen3 team shared the 80B Qwen3-Coder-Next model (parameters active), which made big headlines for outperforming much larger models li DeepSeek V3.2 (37B active) and Kimi K2.5 and GLM-4.7 (both 32B active) on coding

Figure 14: Qwen3-Coder-Next performance on a coding benchmark next to other popular coding models; this figure appeared in the official technical report.

Moreover, as shown in the benchmark figure above, the Qwen3-Coder-Next SWE-Ber Pro performance is roughly on par with Claude Sonnet 4.5 (and only slightly below Cla Opus 4.5), which is impressive for a relatively small open-weight model!

Using the ollama version of Qwen3-Coder-Next locally, the model takes about 48.2 GB storage space and 51 GB of RAM.

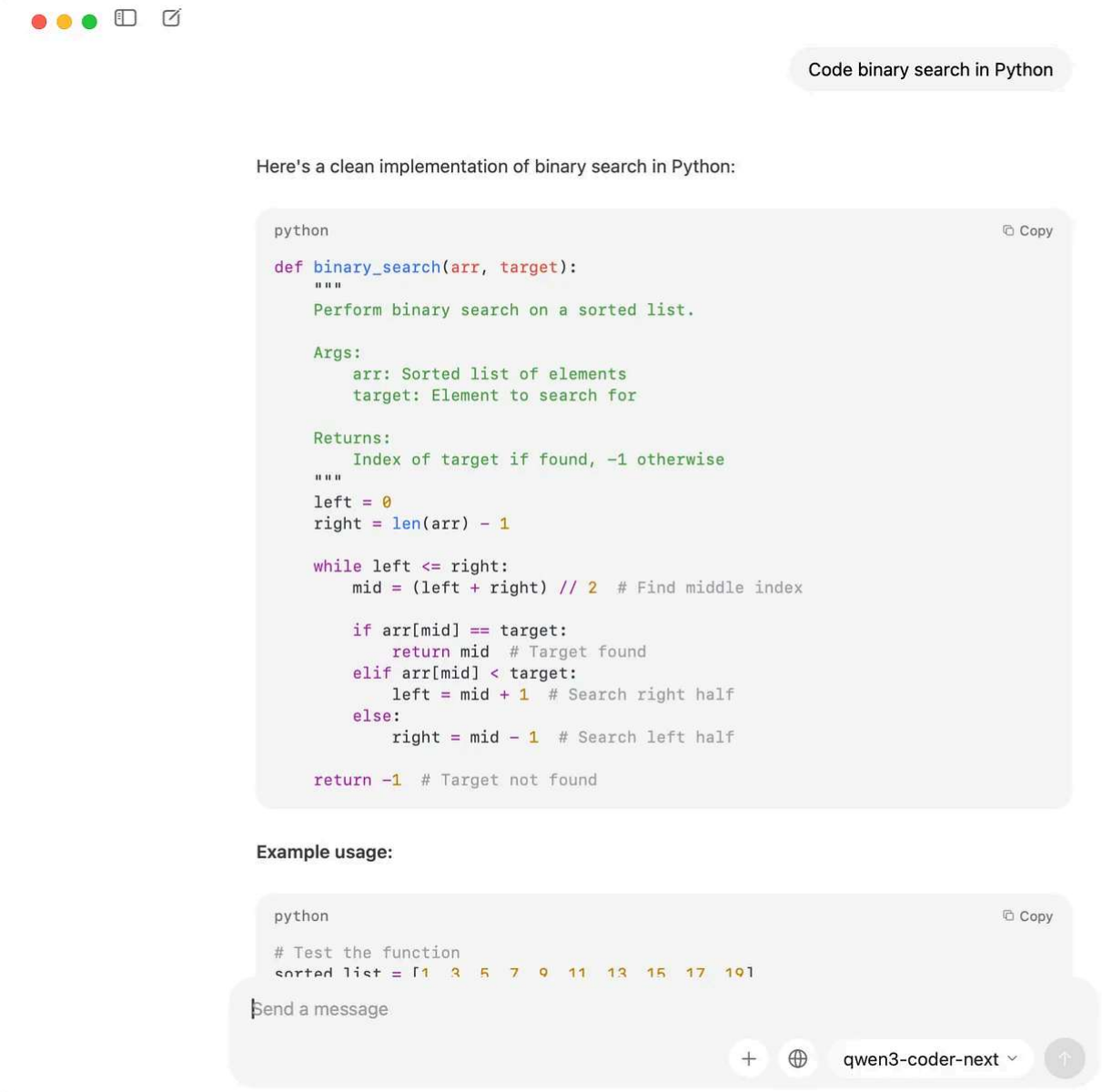


Figure 15: Running Qwen3-Coder-Next locally.

Note that the architecture behind Qwen3-Coder-Next is exactly the same as Qwen3-80B (in fact, the pre-trained Qwen3-Next 80B is used as a base model for further mid- post-training). Figure 16 below shows the Qwen3-Next architecture next to a regular Q 235B model for reference.

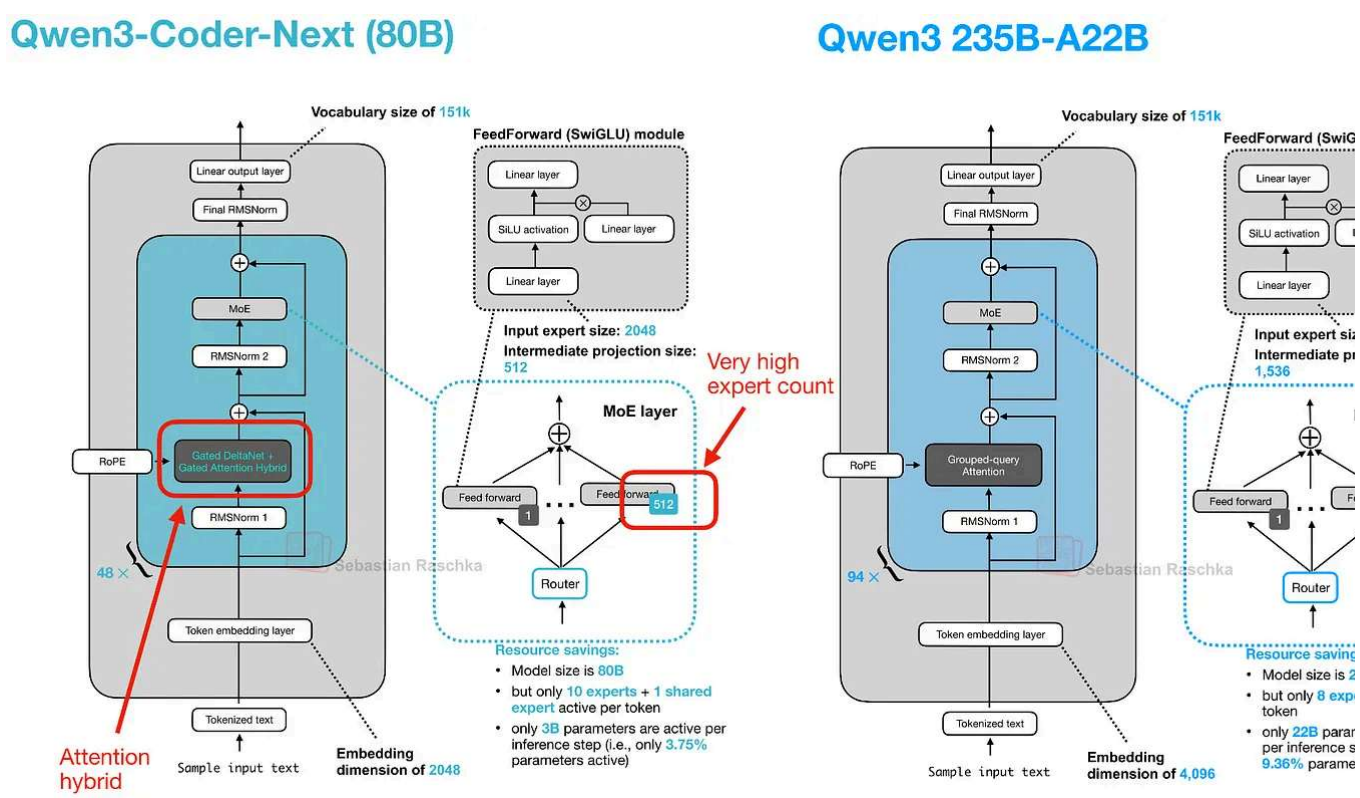


Figure 16: Qwen3-Coder-Next 80B (3B parameters active per token) and the 3x larger Qwen3 235B-A22B architecture.

The new Qwen3 Next architecture stands out because, despite being 3x smaller than t previous 235B-A22B model, it introduces four times as many experts and even adds a shared expert. Both of these design choices (a high expert count and the inclusion of a shared expert).

The other highlight is that they replace the regular attention mechanism with a Gated DeltaNet + Gated Attention hybrid, which helps enable the native 262k token context l in terms of memory usage (the 235B-A22B model supported 32k natively and 131k w YaRN scaling).

So how does this new attention hybrid work? Compared to grouped-query attention (G which is still standard scaled dot-product attention (sharing K/V across query-head gr to cut KV-cache size and memory bandwidth as discussed earlier, but whose decode c and cache still grow with sequence length), their hybrid mechanism mixes Gated Delta blocks with Gated Attention blocks in a 3:1 ratio as shown in Figure 17.

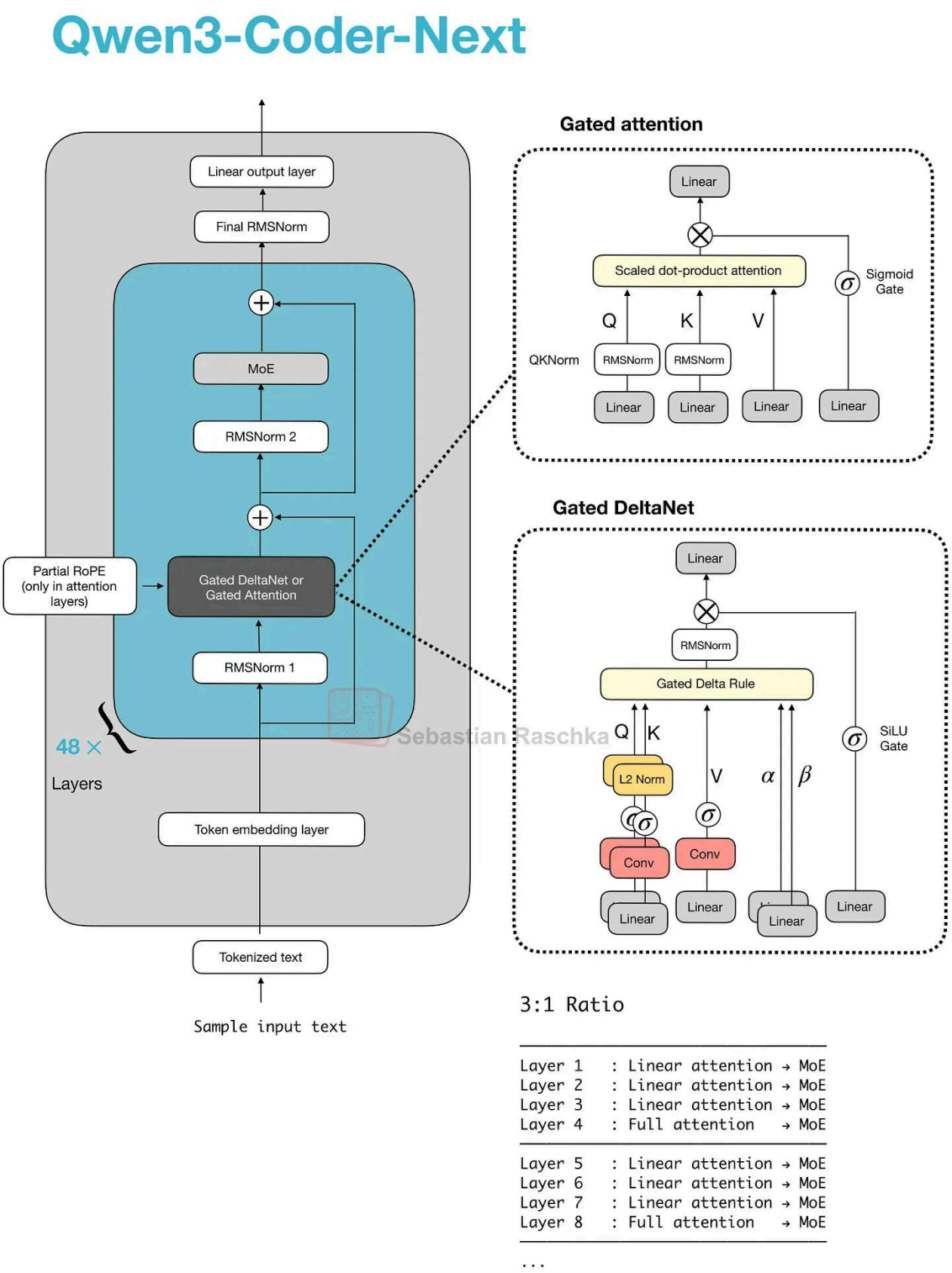


Figure 17: The Qwen3-Coder-Next attention hybrid setup.

We can think of the gated attention block as standard scaled-dot-product attention us GQA, with a few tweaks on top. The main differences between gated attention and plai block are:

1. an output gate (sigmoid-controlled, usually per-channel) that scales the attention before it is added back to the residual;

2. zero-centered RMSNorm for QKNorm, rather than a standard RMSNorm;
3. partial RoPE (on a subset of dimensions).

Note that these are essentially just stability changes to GQA.

The Gated DeltaNet is a more significant change. In the DeltaNet block, q, k, v, and two (α, β) are produced by linear and lightweight convolutional layers with normalization, ar layer replaces attention with a fast-weight delta rule update.

However, the tradeoff is that DeltaNet offers less precise content-based retrieval than attention, which is why one gated attention layer remains.

Given that attention grows quadratically, the DeltaNet component was added to help w memory efficiency. In the "linear-time, cache-free" family, the DeltaNet block is essenti alternative to Mamba. Mamba keeps a state with a learned state-space filter (essential dynamic convolution over time). DeltaNet keeps a tiny, fast-weight memory updated w and β, and reads it with q, using small convolutions only to help form q, k, v, α, β.

For more details on the attention hybrid and Qwen3-Next architecture, please see my previous article Beyond Standard LLMs.

Since this article is primarily focused on LLM architectures, the training details are outs scope. However, interested readers can find more information in their detailed technica report on GitHub.

5. z.AI's GLM-5: A New Flagship Open-Weight Model

The GLM-5 release on February 12th was a big deal, because at the time of its release appeared to be on par with the major flagship LLM offerings, including GPT-5.2 extra-l Gemini Pro 3, and Claude 4.6 Opus. (That said, benchmark performance does not necessarily translate to real-world performance.)

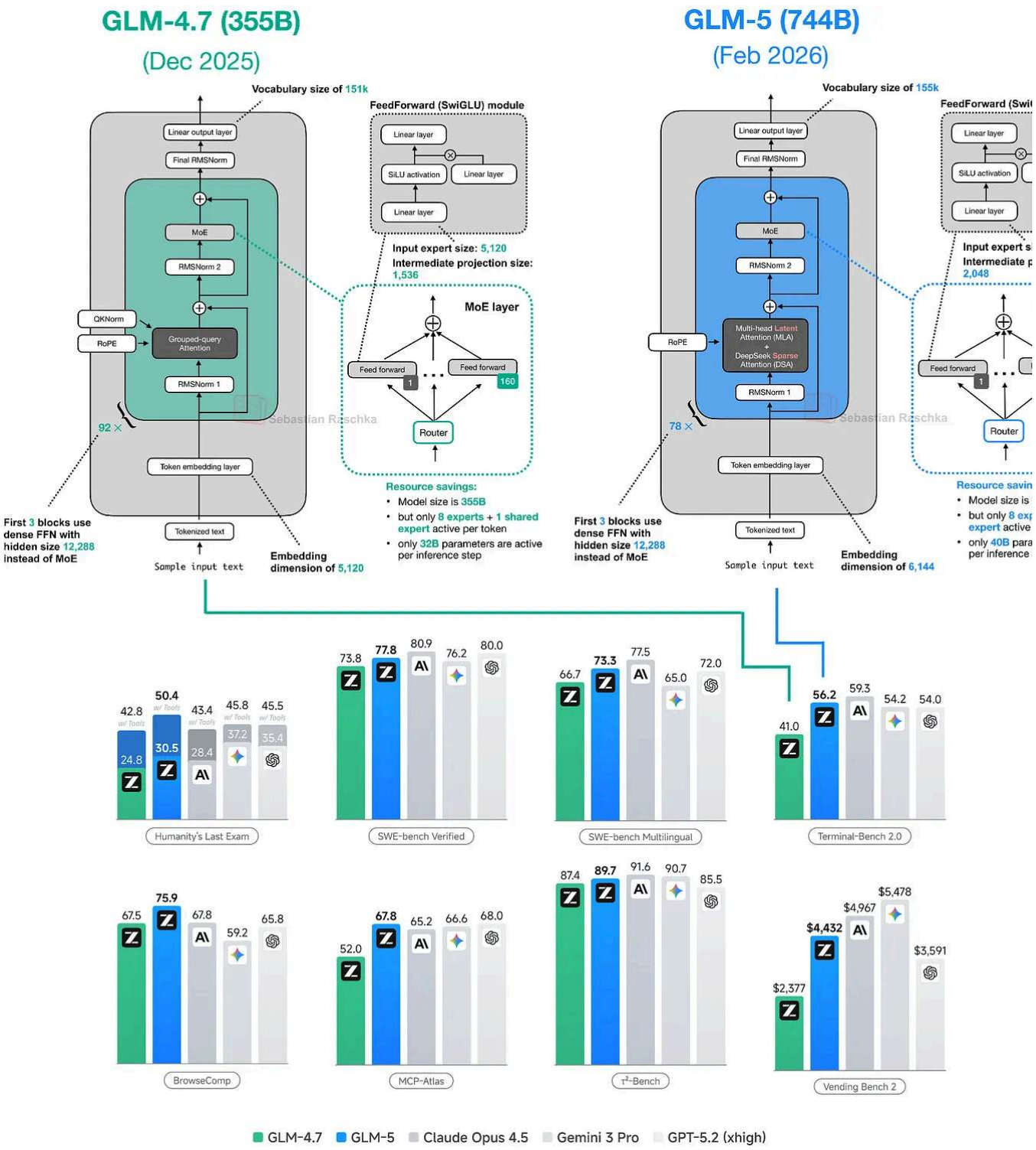


Figure 18: GLM-5 architecture next to its GLM-4.7 predecessor. Benchmarks at the bottom taken from the official GLM-5 technical report.

Not too long ago, GLM-4.7 (December 2025) was one of the strongest open-weight n
GLM-5 shows a major modeling performance improvement based on the benchmark s
in Figure 18 above. That jump is likely partly due to improvements to the training pipelir
likely largely attributed to its 2x larger parameter count from 355B parameters in GLM-

744B parameters in GLM-5. This size increase now places GLM-5 between DeepSeel (671B) and Kimi K2.5 (1T) in terms of scale.

Comparing the benchmark numbers of the previously discussed Kimi K2.5 (1T), the sn GLM-5 (744B) model seems slightly ahead, as shown in the table below.

Benchmark	GLM-5	Kimi K2.5
Humanity's Last Exam (Full)	50.4	50.2
BrowseComp	75.9	74.9
DeepSearchQA	89.7	77.1
SWE-bench Verified	77.8	76.8
SWE-bench Multilingual	73.3	73.0

Figure 19: GLM-5 (744B) and Kimi K2.5 (1T) benchmark performance side by side (larger is better).

Like GLM-4.7, all the other models discussed so far, GLM-5 is a Mixture-of-Experts m The number of active parameters per token increases only slightly, from 32B in GLM-4 40B in GLM-5.

As shown in Figure 20 below, GLM-5 now adopts DeepSeek's multi-head latent attent well as DeepSeek Sparse Attention. (I described DeepSeek Sparse Attention in more d From DeepSeek V3 to V3.2: Architecture, Sparse Attention, and RL Updates.)

These modifications are likely intended to reduce inference costs when working with lo contexts. Otherwise, the overall architecture remains relatively similar.

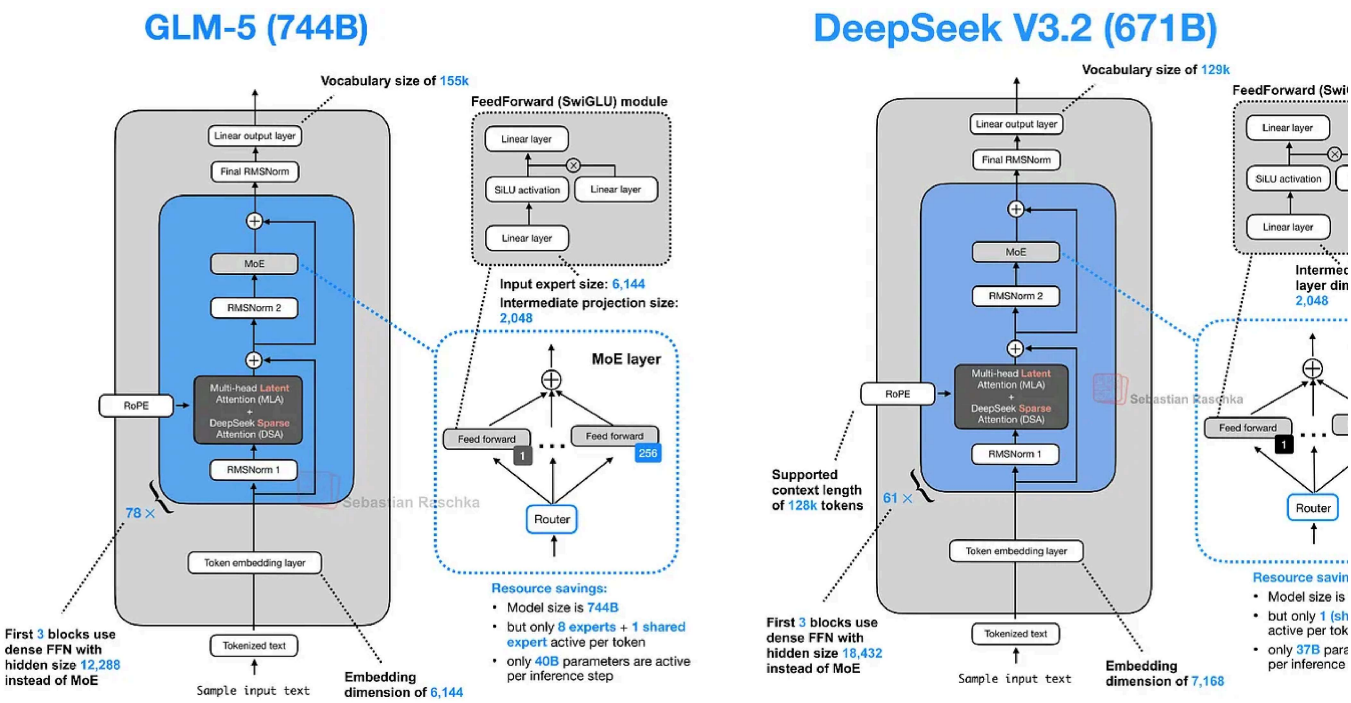


Figure 20: GLM-5 and DeepSeek V3.2 side by side (two similar architectures at a similar size).

The increase in total size over GLM-4.7 mainly comes from expanding the number of experts, from 160 (GLM-4.7) to 256 (GLM-5), and slightly increasing layer dimensions (while keeping the number of experts the same at 8 regular + 1 shared expert per token example, the embedding dimension and expert size increase from 5,120 to 6,144, and intermediate projection size rises from 1,536 to 2,048.

Interestingly, the number of transformer layers is reduced from 92 in GLM-4.7 to 78 in 5. I assume this change is also intended to reduce inference costs and improve latency layer depth cannot be parallelized in the same way as width.

Additionally, I also checked an independent benchmark (here, the hallucination leaderb and it indeed looks like GLM-5 is on par with Opus 4.5 and GPT-5.2 (while using fewe tokens).

Benchmark	GLM-5	Kimi K2.5	Claude Opus 4.5	Gemini 3 Pro	GPT-5.2 (x
Humanity's Last Exam (Full)	50.4	50.2	43.4	45.8	45.5
BrowseComp	75.9	74.9	67.8	59.2	65.8
DeepSearchQA	89.7	77.1			
SWE-bench Verified	77.8	76.8	80.9	76.2	80.0
SWE-bench Multilingual	73.3	73.0	77.5	65.0	72.0
	Lower is better				
Hallucination Rate	10.1 %	14.2 %	10.3 %	13.6 %	10.8 %
Factual Consistency Rate	89.9 %	85.8 %	89.7 %	86.4 %	89.2 %
Answer Rate	99.7 %	92.2 %	98.6 %	99.4 %	100.0 %
Average Summary Length (Words)	74.4	112.0	145.8	101.9	186.3

Figure 21: Next to the overall benchmark performance, this table adds hallucination rates from the hallucination leaderboard.

Furthermore, looking at the most recent Artificial Intelligence Index, which aggregates various benchmarks, GLM-5 is indeed slightly ahead of Kimi K2.5 and only one point b GPT-5.2 (xhigh) and the recent Claude Sonnet 4.6.

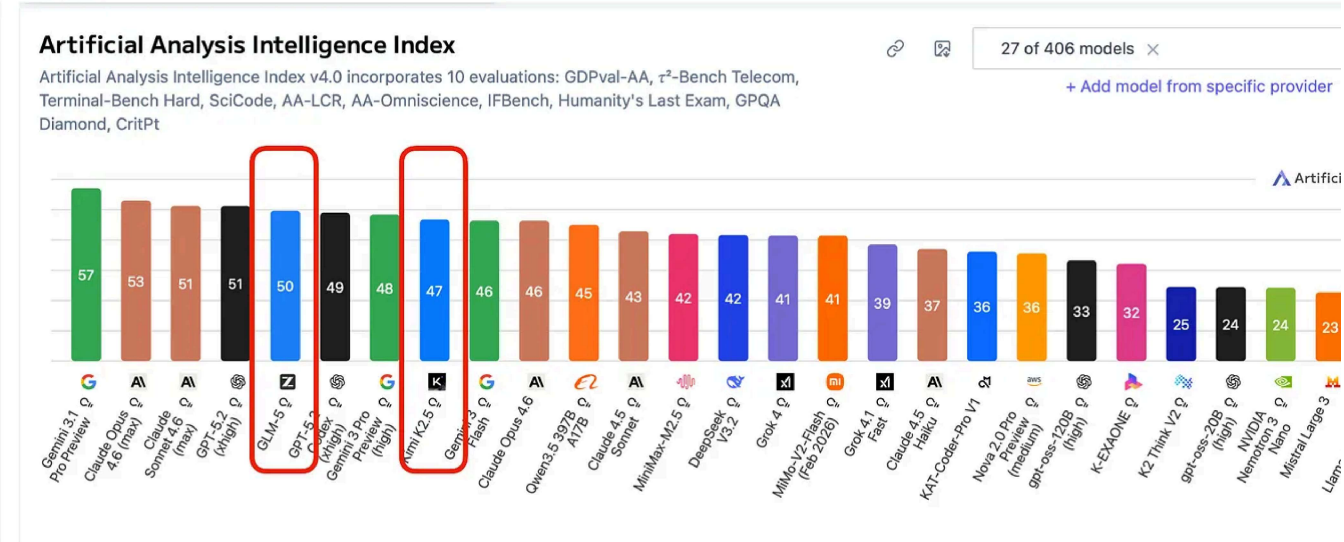


Figure 22: Artificial Intelligence Index snapshot from Feb 21, 2026.

6. MiniMax M2.5: A Strong Coder with "Only" 230B Parameters

The aforementioned GLM-5 and Kimi K2.5 are popular open-weight models, but acco to OpenRouter statistics, they pale in comparison to MiniMax M2.5, which was release February 12 as well.

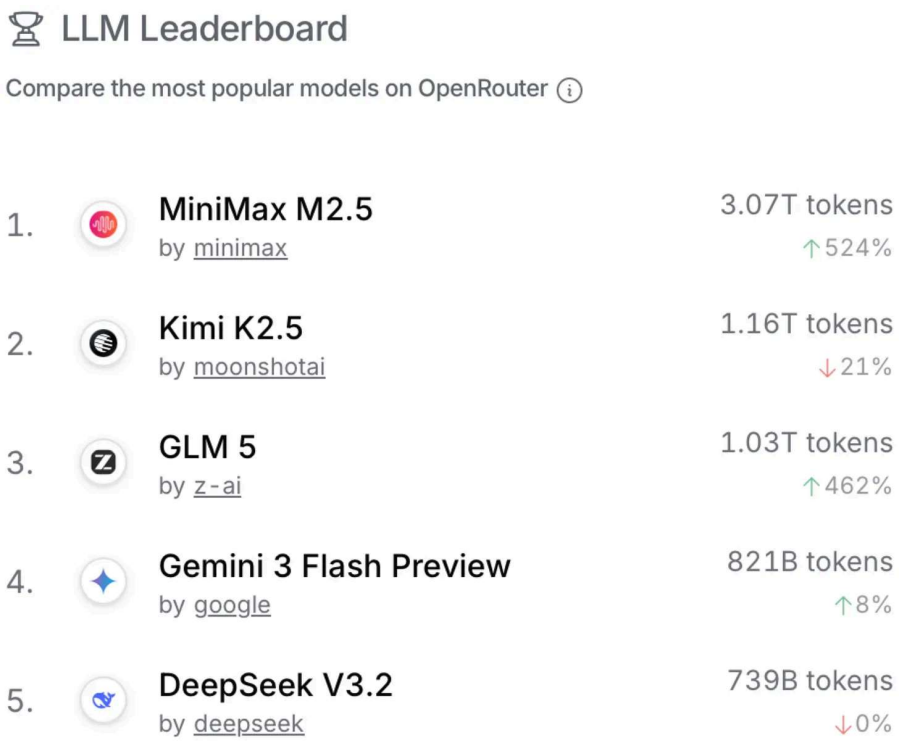


Figure 23: OpenRouter usage snapshot from Feb 21, 2026.

OpenRouter is a platform and API that lets developers access and route requests acros many different LLMs from various providers. Note that while its usage statistics are a g indicator of open-weight model popularity, it's heavily biased towards open-weight mc (versus proprietary models), since most users use proprietary models through the offic platform directly. There is also usage bias across open-weight models, since many pec also use open-weight models through the official developers' APIs. Anyways, it can stil interesting place to guesstimate the relative popularity of open-weight models that are large to run locally for most users.

Now, back to MiniMax M2.5. Pulling together the GLM-5 data from the SWE-Bench Ve coding benchmark and combining it with the reported MiniMax M2.5, the latter appear

a slightly stronger model (at least when it comes to coding).

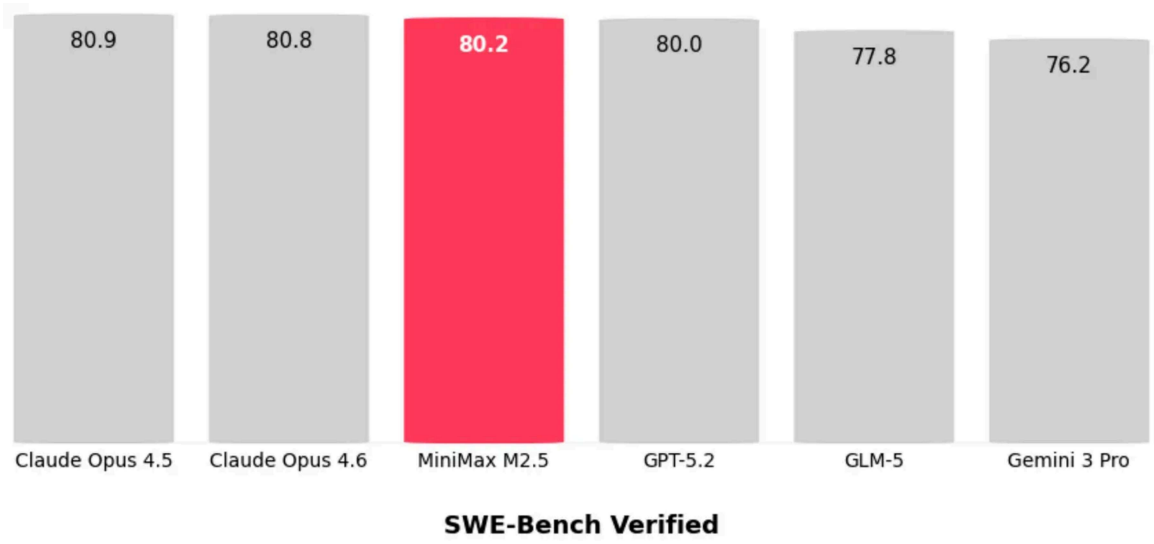


Figure 24: MiniMax M2.5 coding performance on SWE-Bench Verified

Side note: It's interesting to see Opus 4.5 and Opus 4.6 practically scoring identically c SWE-Bench Verified. This can be an indicator that LLM progress has stalled. I don't thi that's true, though, given that users of Opus 4.6 can confirm that this model does seen perform better in real-world usage. So, the more likely issue here is that the SWE-Benc Verified benchmark has saturated, and it may no longer be a meaningful benchmark to from now on (in favor of other benchmarks like SWE-Bench Pro, for example). With sat I mean that it potentially contains unsolvable problems due to design issues (as discus a recent Reddit thread and the new "Why SWE-bench Verified no longer measures fro coding capabilities" article by OpenAI).

Anyways, back to the topic of MiniMax M2.5 performance. Looking across a broader selection of benchmarks, according to the Artificial Intelligence Index aggregation, GL remains ahead. This is perhaps no surprise because GLM-5 is still a 4x larger model th M2.5, even though the tokens/sec throughput is quite similar.

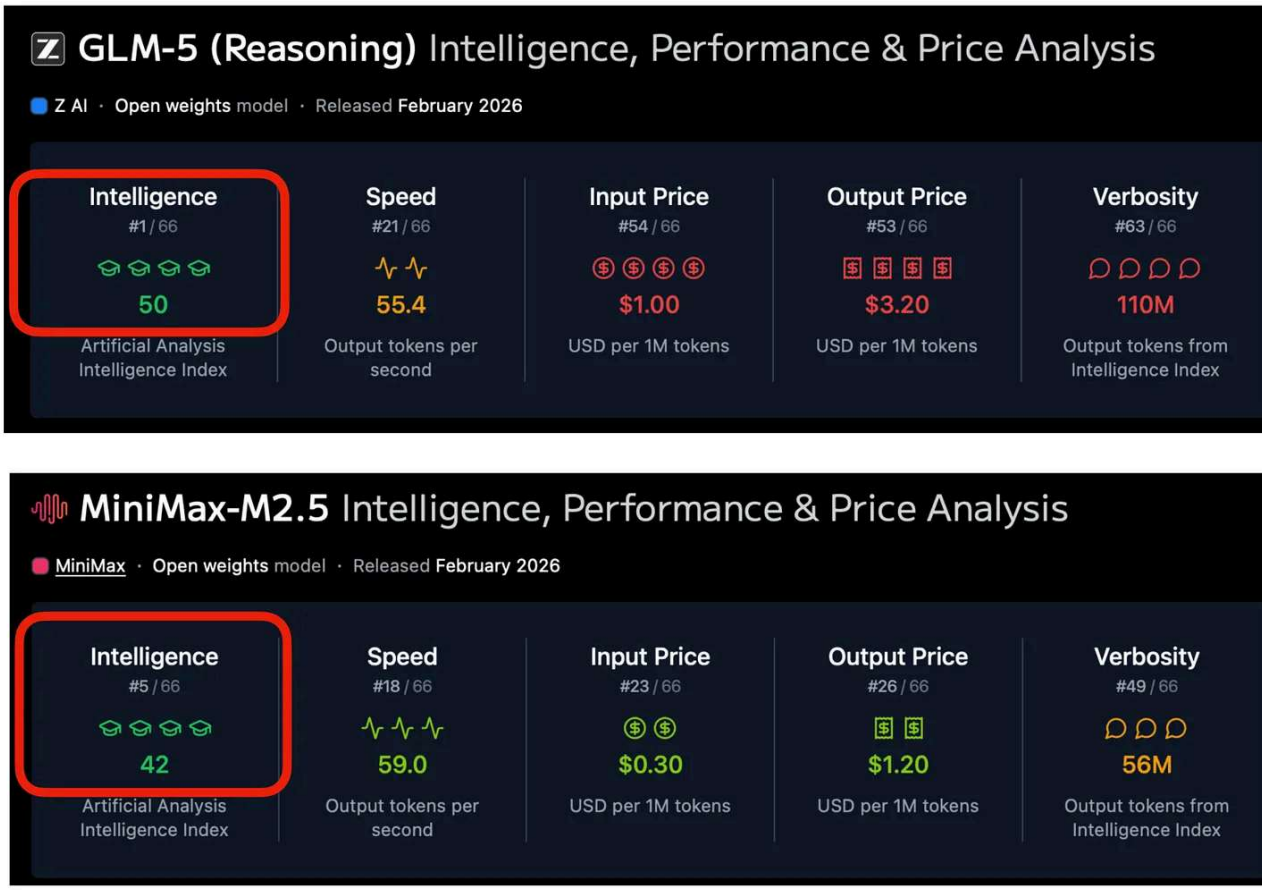


Figure 25: GLM-5 vs MiniMax M2.5 comparison based on the Artificial Intelligence Index (Feb 21, 2026)

I think MiniMax M2.5's popularity is partly owed to the fact that it is a smaller, cheaper r with roughly similar modeling performance (i.e., a good bang for the buck).

Architecture-wise, MiniMax M2.5 is a 230B model with a fairly classic design: just plair Grouped Query Attention, no sliding window attention or other efficiency improvement

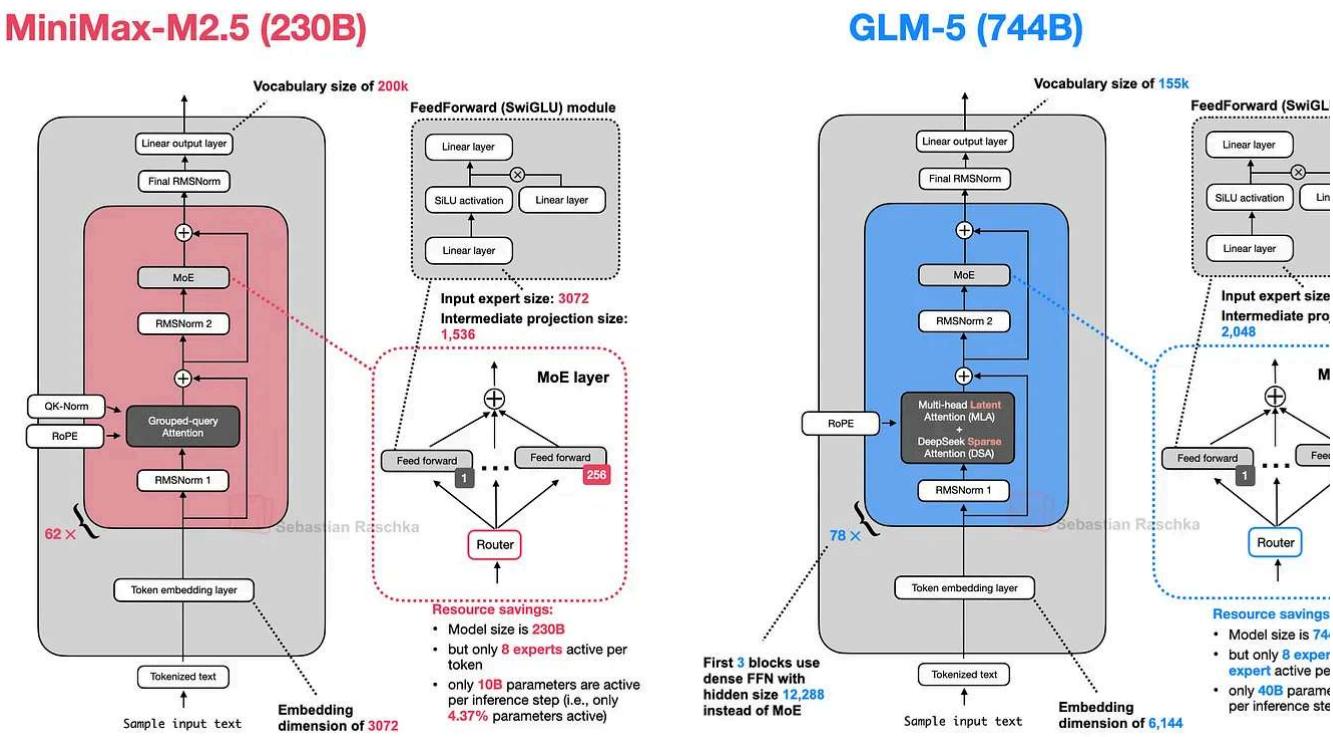


Figure 26: MiniMax M2.5 next to GLM-5.

So far, this is also the first architecture in this report that doesn't come with a detailed technical report, but you can find additional information on the model hub page.

7. Nanbeige 4.1 3B: A Strong Llama 3 Successor

In this section, we are switching gears and finally covering a smaller model that can rur locally on a laptop. But first let's start with some context before we get to Nanbeige 4.1

Qwen models have always been very popular models. I often tell the story that when I v advisor during the NeurIPS LLM efficiency challenge a few years back, most of the win solutions were based on a Qwen model.

Now, Qwen3 is likely among the most widely used open-weight model suite since they such a wide range of sizes and use cases (from 0.6B to 235B)

Especially the smaller models (80B and less, like Qwen3-Next, covered previously) are for local use on consumer hardware.

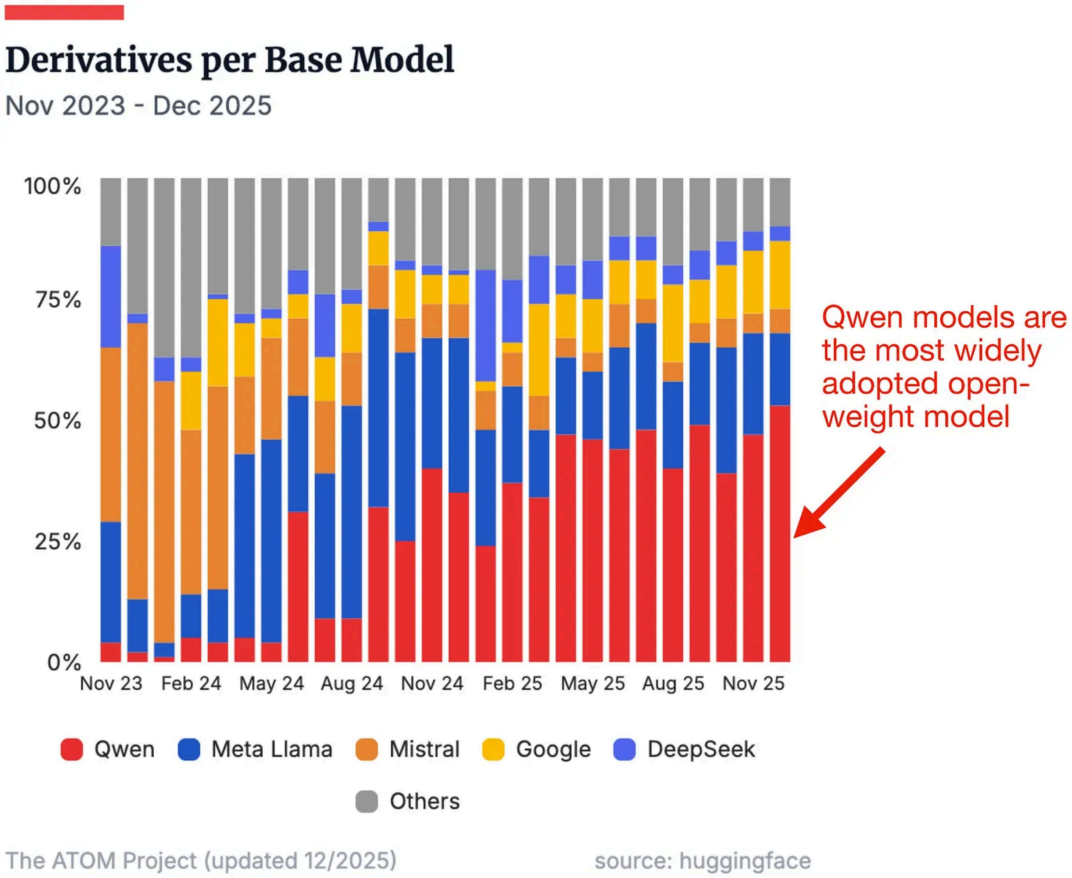


Figure 27: Relative adoption popularity of open-weight models. Note that this shows the number of models on the Hugging Face model hub that are finetuned using one of those models as a base model. (This is not the number of people who use the models on their computer locally, which would be a number impossible to know.) Source: Atom Project.

Why I am mentioning all this is that Nanbeige 4.1 3B seems to target the "small" LLM o device use case that Qwen3 is so popular for. According to the Nanbeige 4.1 3B benchmarks, their model is way ahead of Qwen3 (perhaps no surprise, given that Qwe almost a year old).

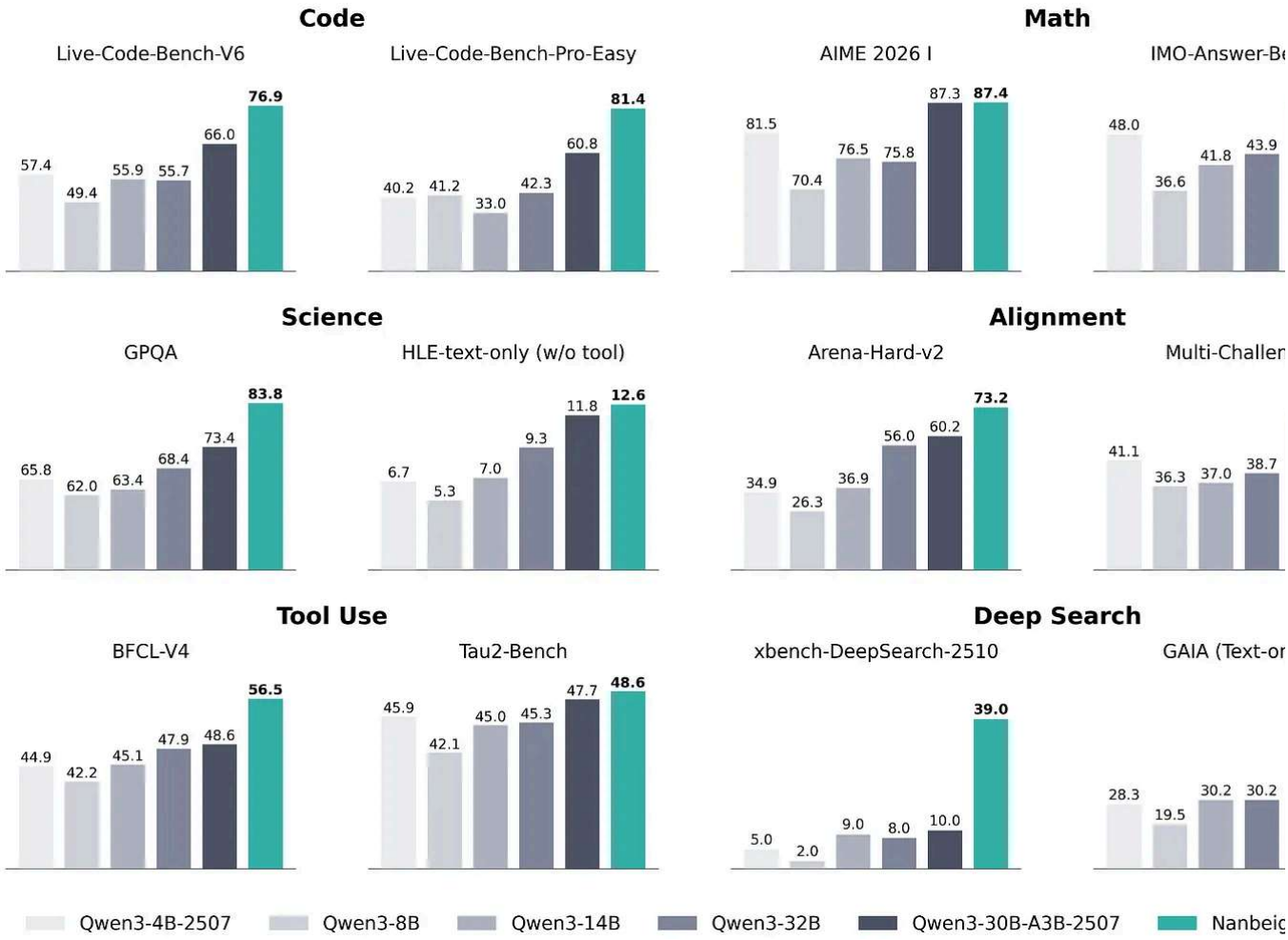


Figure 28: Nanbeige 4.1 3B benchmark comparison with Qwen3 (Source: Nanbeige 4.1 3B model hub page).

Architecture-wise, Nanbeige 4.1 3B is similar to Qwen3 4B, which is, in turn, very simila Llama 3.2 3B. I am showing Nanbeige 4.1 3B next to Llama 3.2 3B below because it is most similar in size.

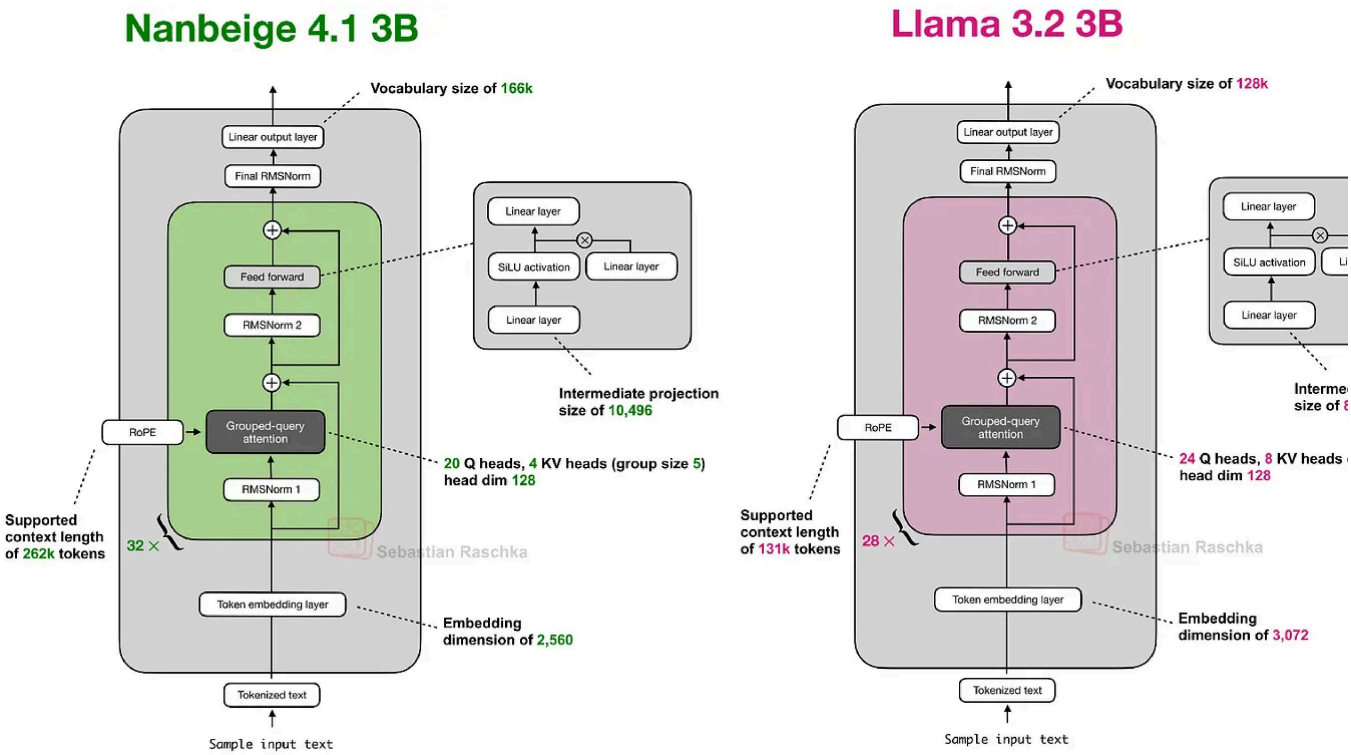


Figure 29: Nanbeige 4.1 3B next to Llama 3.2 3B.

Nanbeige 4.1 3B uses the same architectural components as Llama 3.2 3B, with some scaling differences (slightly smaller embedding dimensions and larger intermediate projections, and so on). The one difference not shown in the figure above is that Nanbe does not tie the input embedding weights to the output layer weights, whereas Llama 3 does. (In my experience, weight tying is a nice way to reduce the total number of param but it almost always results in worse training performance as evidenced by higher train and validation losses.)

As mentioned before, this article focuses primarily on the architecture comparisons. Ar this case, most of the performance gains (compared to the Nanbeige 4 3B predecesso come from additional post-training with supervised fine-tuning and reinforcement lear but interested readers can find more information in the detailed technical report.

8. Qwen3.5 and the Continuation of Hybrid Attention

While the previous section briefly covered Qwen3 as the most open-weight model fam is getting a bit long in the tooth as its release is almost a year ago (if we don't count the

Qwen3-Next variants geared towards efficiency). However, the Qwen team just releas new Qwen3.5 model variant on February 15.

Qwen3.5 397B-A17B, a Mixture-of-Experts (MoE) with 397B parameters (17B active token), is a step up from the largest Qwen3 model, which is 235B parameters in size. (is also the 1 trillion-parameter Qwen3-Max model, but it was never released as an ope weight model.)

The obligatory benchmark overview shows that Qwen3.5 exceeds the previous Qwen model across the board, with a much stronger focus on agentic terminal coding applic (the main theme this year). Qwen3.5 appears to be roughly on par with GLM-5 and Mi M2.5 in terms of pure agentic coding performance (e.g., SWE-Bench Verified).

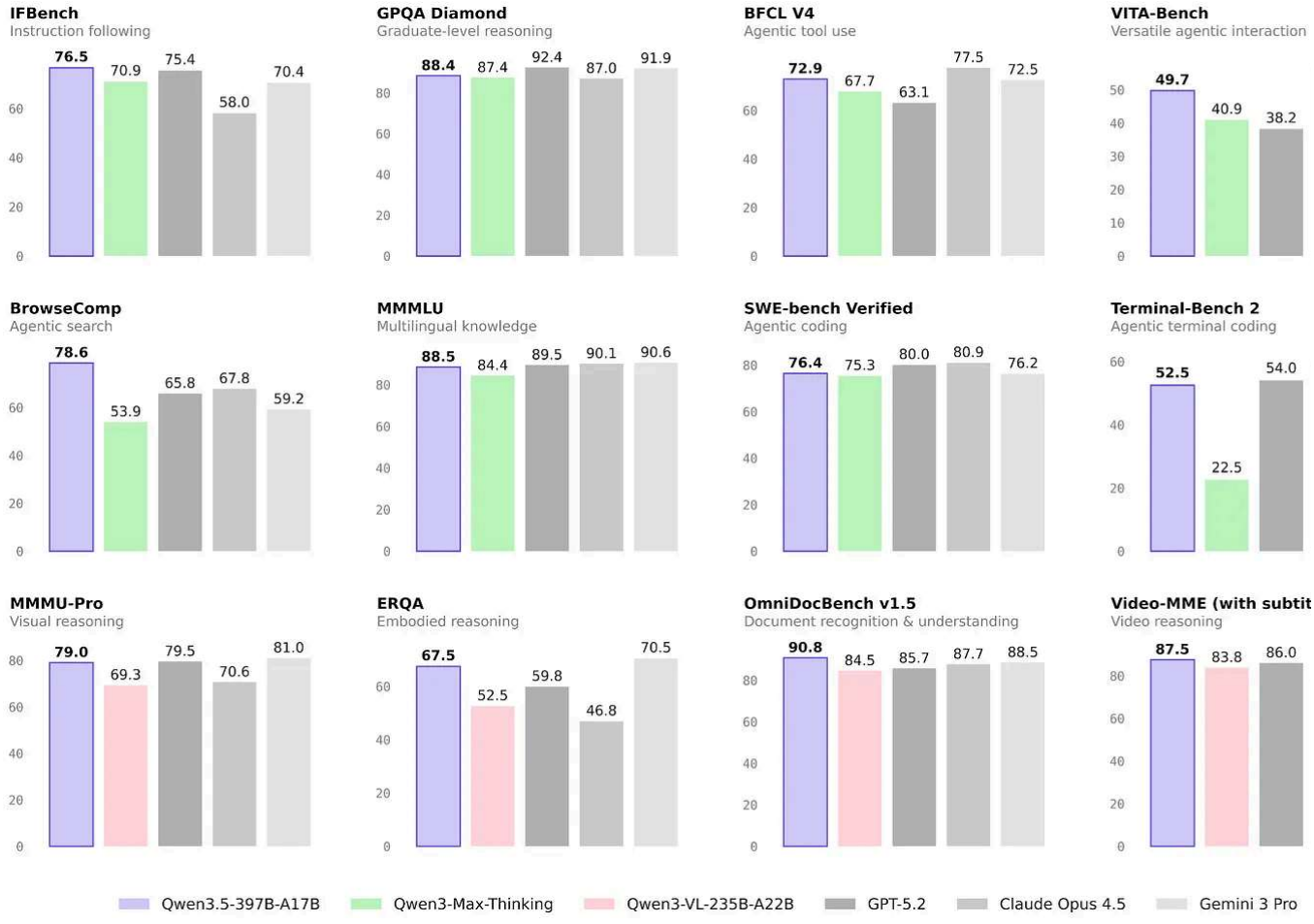


Figure 30: Qwen3.5 benchmark overview from the official model hub page.

Since the Qwen team likes to release a separate coding model (e.g., see Qwen3-Code which we discussed previously), this makes me curious to see how a potential Qwen3. Coder will perform.

Architecture-wise, Qwen3.5 adopts the hybrid attention model (featuring Gated Deltal that Qwen3-Next and Qwen3-Coder-Next (section 4) used. This is interesting becaus Qwen3-Next models were initially an alternative to the full-attention Qwen3 models, b suggests that the Qwen team has now adopted the hybrid attention mechanism into it line of models.

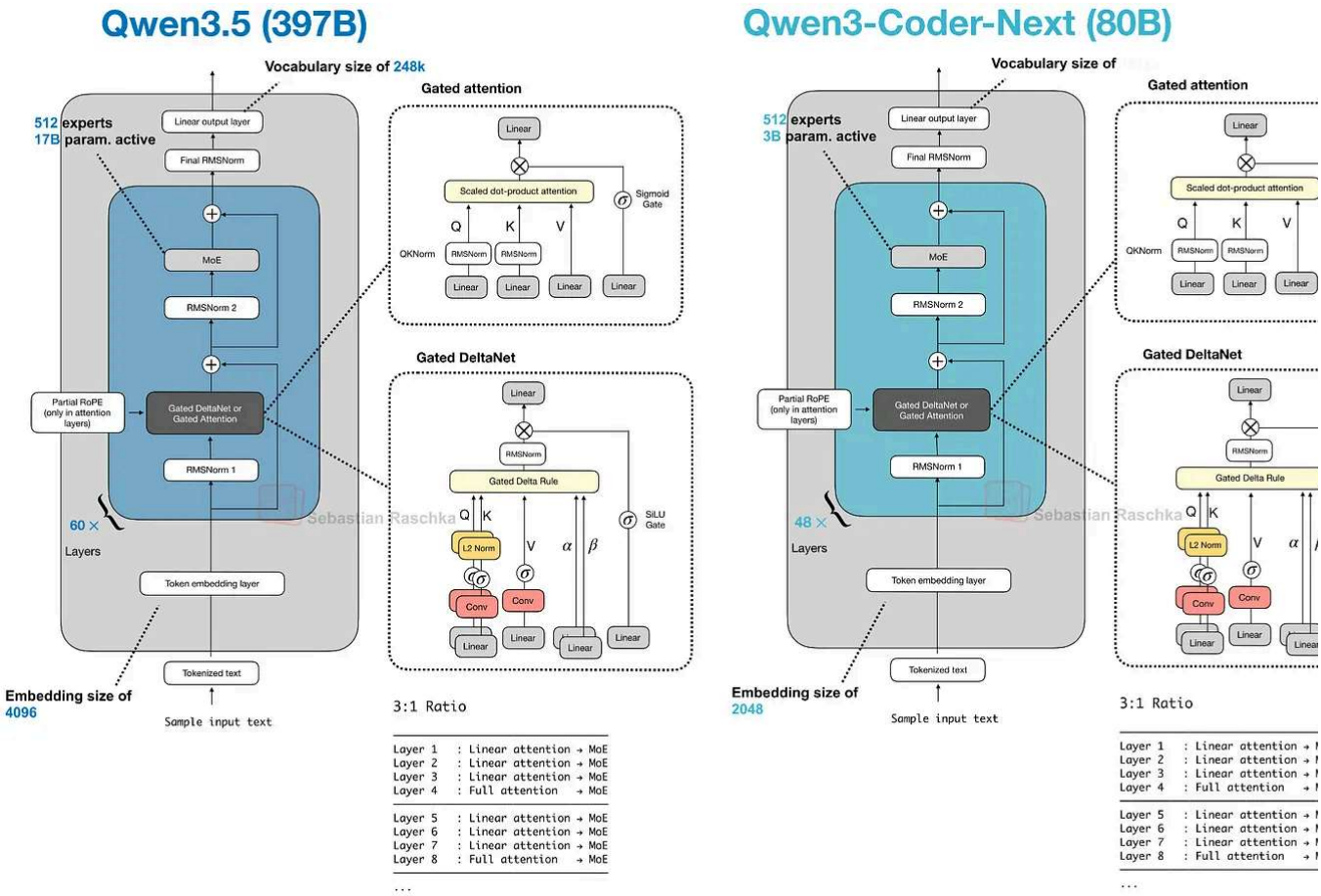


Figure 31: Comparison between Qwen3.5 and the Qwen3(-Coder)-Next architectures.

Besides scaling up the model size, as shown in the figure above, Qwen3.5 now also inc multimodal support (previously, it was only available in separate Qwen3-VL models).

Anyways, Qwen3.5 is a nice refresh of the Qwen series, and I hope that we will see sm Qwen3.5 variants in the future, too!

Edit: Just as I finalized this article, the Qwen team launched said smaller model variants:

- Qwen3.5-27B

- Qwen3.5-35B-A3B
- Qwen3.5-122B-A10B

9. Ant Group's Ling 2.5 1T with Lightning Attention

Ling 2.5 (and the reasoning variant Ring 2.5) are 1-trillion-parameter LLMs with a hybr attention architecture in a similar spirit to Qwen3.5 and Qwen3-Next.

However, instead of Gated DeltaNet, they use a slightly simpler recurrent linear attentic variant called Lightning Attention. In addition, Ling 2.5 adopts the Multi-Head Latent Attention (MLA) mechanism from DeepSeek.

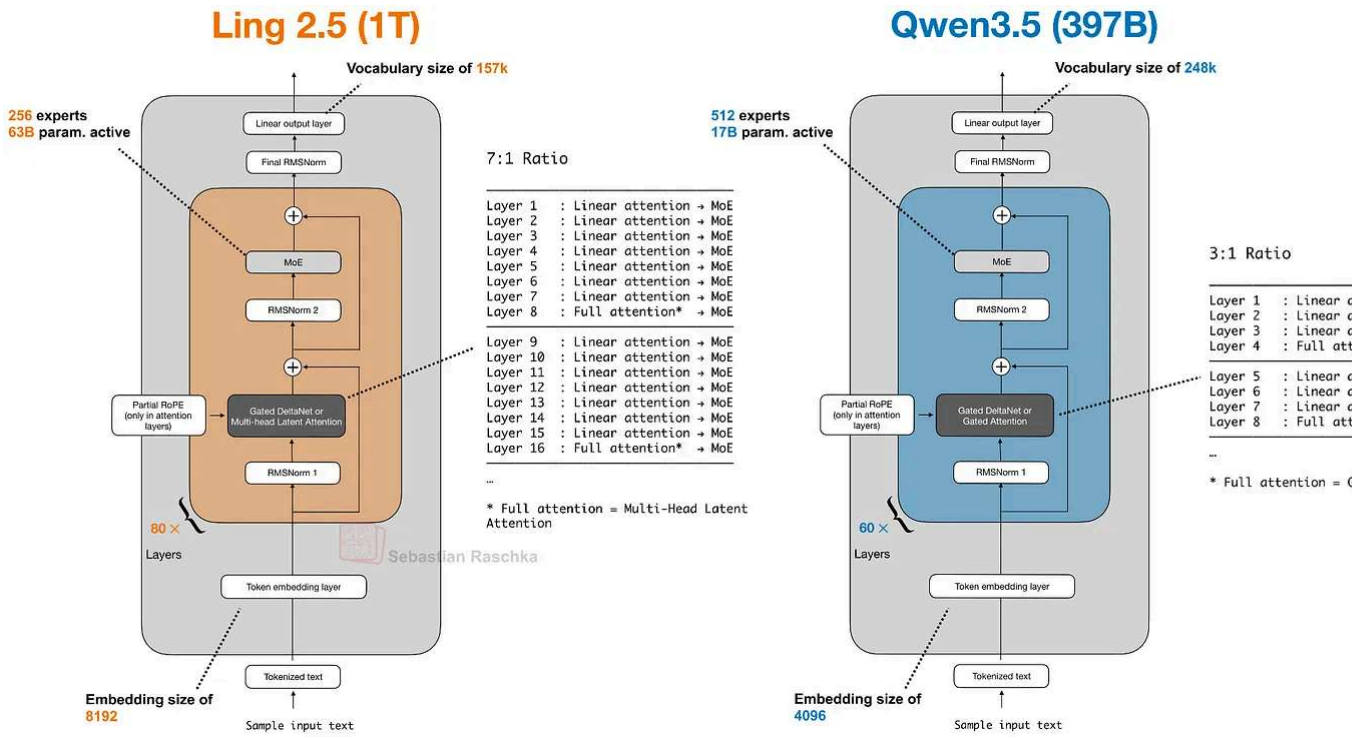


Figure 32: Ling 2.5 compared to Qwen3.5; both architectures are linear attention hybrids.

Ling 2.5 is not the strongest model in terms of absolute benchmark performance, but it selling point is very good efficiency in long contexts (due to the hybrid attention). Unfortunately, there are no direct comparisons to Qwen3.5, but compared to Kimi K2 (parameters; the same size as Ling 2.5), Ling 2.5 achieves a 3.5x higher throughput at a sequence length of 32k tokens.

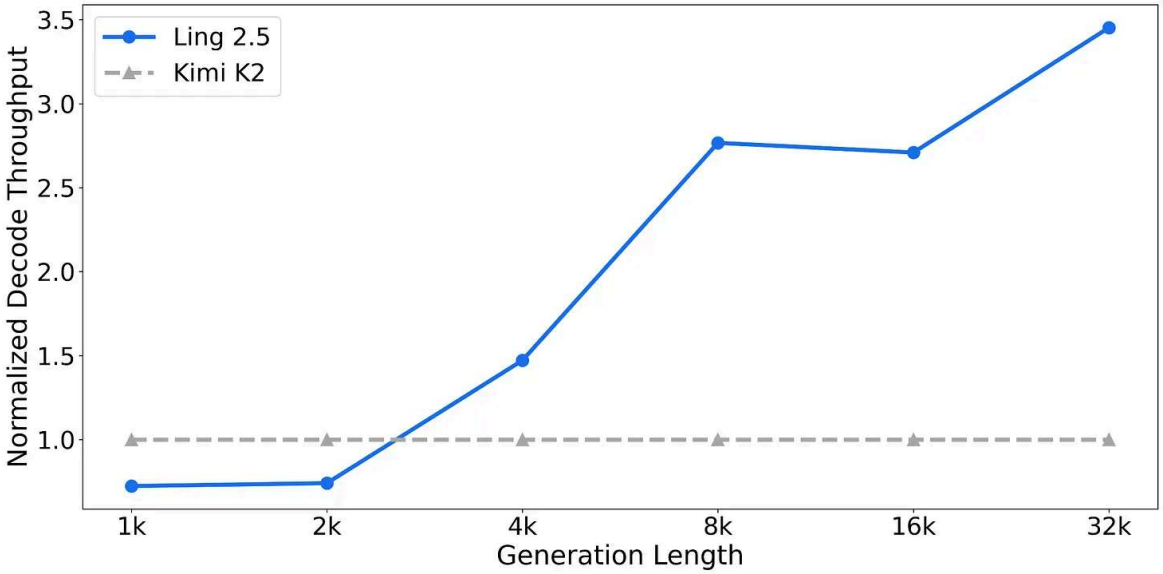


Figure 33: Relative throughput of Ling 2.5 compared to Kimi K2 (same 1 trillion parameter size); note that the throughput is normalized so that Kimi K2 is shown at 1x (Kimi's throughput is not linear even though it appears linear in this plot). Source: Ling 2.5 model hub page.

10. Tiny Aya: A 3.35B Model with Strong Multilingual Support

Released on February 17, Tiny Aya is a new, "small" LLM by Cohere that is said to be th "most capable multilingual open-weight model" at the 3B parameter size class. (Tiny A outperforms Qwen3-4B, Gemma 3 4B, and Ministral 3 3B according to the announcer post).

This is a great model to run and experiment with locally. The only caveat is that while it' open-weight model, its licensing terms are relatively restricted and only allow non-commercial use.

That aside, Aya is a 3.35B parameter model that comes in several flavors that are usefu

personal and (non-commercial) research use:

- tiny-aya-base (base model)
- tiny-aya-global (best balance across languages and regions)
- tiny-aya-fire (optimized for South Asian languages)

- tiny-aya-water (optimized for European and Asia Pacific languages)
- tiny-aya-earth (optimized for West Asian and African languages)

More specifically, below is a list of languages the models are optimized for.

Region	Languages	Op M
Asia Pacific	Traditional Chinese, Cantonese, Vietnamese, Tagalog, Javanese, Khmer, Thai, Burmese, Malay, Korean, Lao, Indonesian, Simplified Chinese, Japanese	tiny wa
Africa	Zulu, Amharic, Hausa, Igbo, Swahili, Xhosa, Wolof, Shona, Yoruba, Nigerian Pidgin, Malagasy	tiny ear
South Asia	Telugu, Marathi, Bengali, Tamil, Hindi, Punjabi, Gujarati, Urdu, Nepali	tiny fire
Europe	Catalan, Galician, Dutch, Danish, Finnish, Czech, Portuguese, French, Lithuanian, Slovak, Basque, English, Swedish, Polish, Spanish, Slovenian, Ukrainian, Greek, Bokmål, Romanian, Serbian, German, Italian, Russian, Irish, Hungarian, Bulgarian, Croatian, Estonian, Latvian, Welsh	tiny wa
West Asia	Arabic, Maltese, Turkish, Hebrew, Persian	tiny ear

Figure 34: Languages supported by the various Aya models.

Architecture-wise, Tiny Aya is a classic decoder-style transformer with a few notewort modifications (besides the obvious ones like SwiGLU and Grouped Query Attention), a illustrated in the figure below.

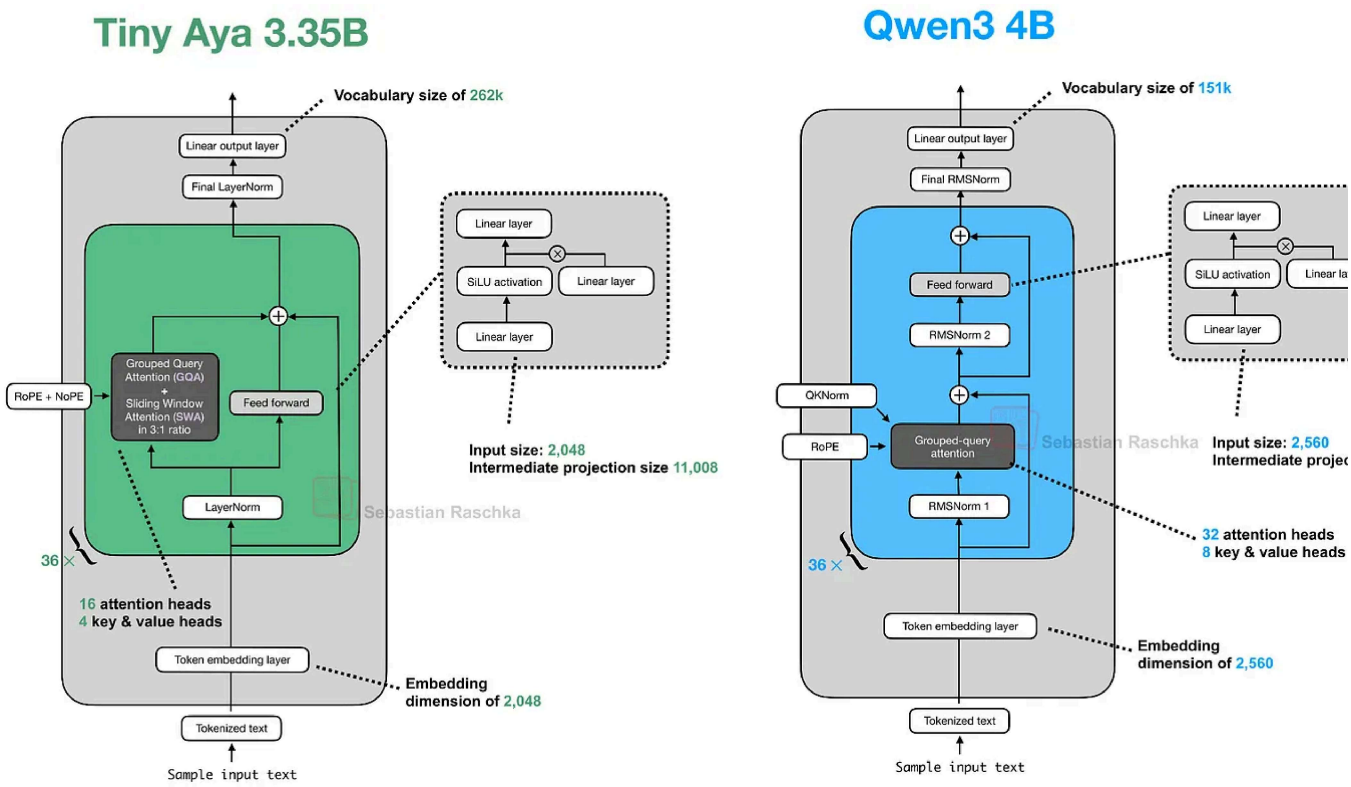


Figure 35: Tiny Aya (featuring a parallel transformer block) and Qwen3 4B side by side.

Overall, the most noteworthy highlight in this architecture is the parallel transformer blo Here, the parallel transformer block computes attention and an MLP from the same normalized input, then adds both to the residual in a single step. I assume this is to redu serial dependencies inside a layer to improve computational throughput.

For those readers familiar with Cohere's Command-A architecture, Tiny Aya seems to b smaller version of it. Also, an interesting detail is that the Tiny Aya team dropped QK-No (an RMSNorm applied to keys and queries inside the attention mechanism); QK-Norm become quite standard for improving training stability in terms of reducing loss spikes. According to a developer on the Cohere team, QK-Norm was dropped "since it can inte with long context performance."

As you may know, I occasionally code architectures from scratch. Since I found the par transformer block quite intriguing and the model runs fine on low-end hardware, I implemented it from scratch (for educational purposes), which you can find here on Gi

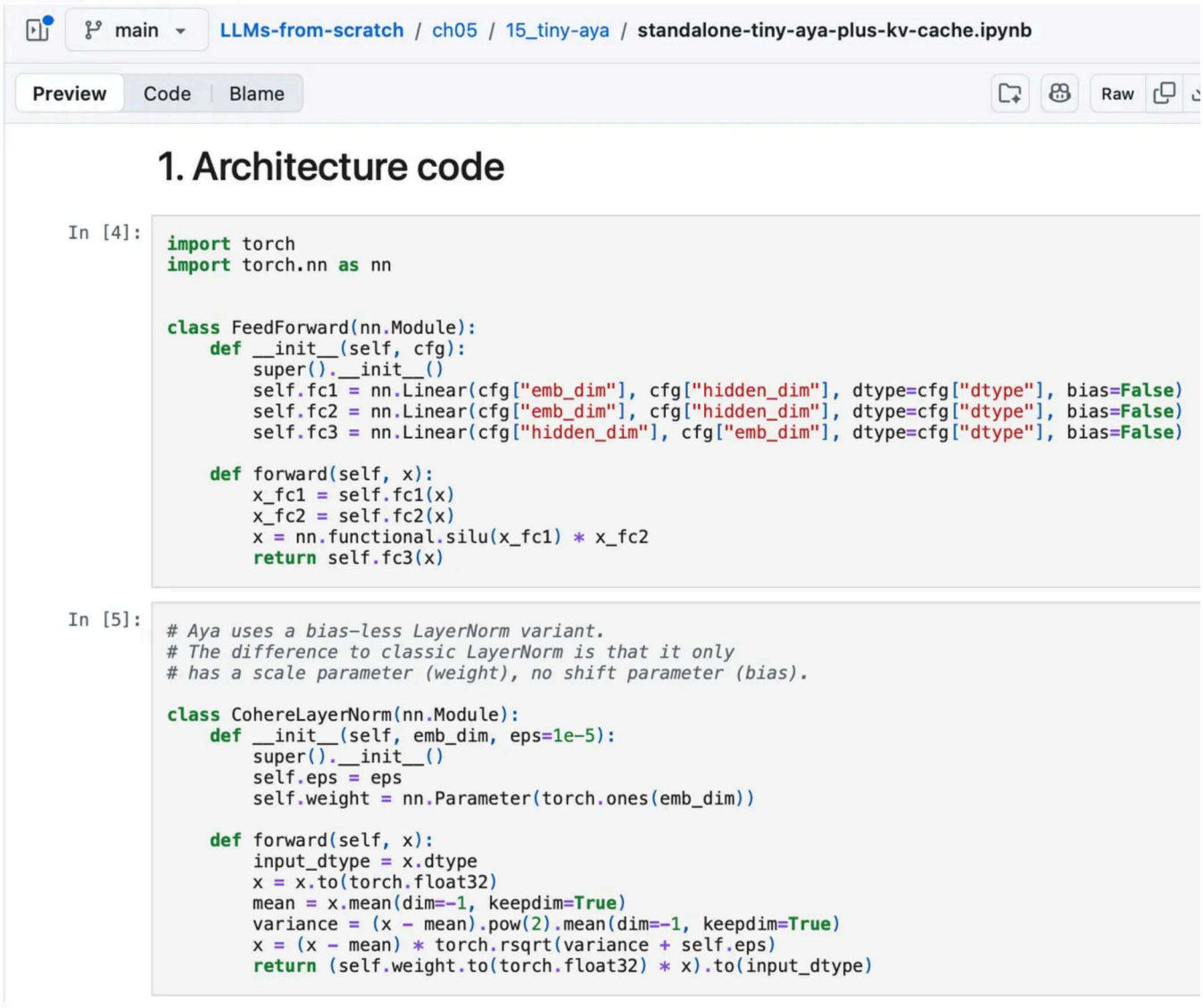


Figure 36: Tiny Aya from-scratch implementation.

Conclusion

This article was quite the whirlwind tour covering the main open-weight LLM releases around February 2026. If there is a takeaway from this, it's that there are various model architectures (all derived from the original GPT model) that work well. Modeling perforr is likely not attributed to the architecture design itself but rather the dataset quality and training recipes (a good topic for a separate article).

That said, architectural design remains an essential part of building a successful LLM, a many developers seem to be steering towards adding more and more computational

performance tweaks. For example, this includes adapting MLA (Kimi K2.5, GLM-5, Ling and DeepSeek Sparse Attention (GLM-5) to continue the Gated DeltaNet (Qwen3.5) o similar forms of linear attention (Ling 2.5).

Attention variant	Models
Only Grouped Query Attention (GQA)	GLM-4.5 GLM-4.7 MiniMax-M2.5 Qwen3 Llama 3.2 3B Nanbeige 4.1
Grouped Query Attention (GQA) + Sliding Window Attention (SWA)	Arcee AI Trinity Large Step 3.5 Flash Tiny Aya Gemma
Multi-Head Latent Attention (MLA)	DeepSeek V3/R1 Kimi K2 Kimi K2.5
Multi-head Latent Attention (MLA) + DeepSeek Sparse Attention (DSA)	DeepSeek V3.2 GLM-5
Gated DeltaNet & Gated Attention	Qwen3.5 Qwen3-Coder-Next
Lightning Attention + MLA	Ling 2.5 & Ring 2.5

Figure 37: Attention types used by the various architectures mentioned in this article.

Also, more classic efficiency tweaks like grouped query attention and sliding window attention (Arcee Trinity, Step 3.5 Flash, Tiny Aya) remain popular. Among the new relea only MiniMax M2.5 and Nanbeige 4.1 stayed very classic here, using only Grouped Qu Attention without any other efficiency tweak.

DeepSeek V4

DeepSeek V4 is the model everyone is waiting for. Unfortunately, as of this writing, it ha been released yet. However, I plan to add it to this article once it's released, which is like

or before the first week of March.

Another interesting model is Sarvam (30B & 100B) from India. The model was recently announced, but it hasn't been released yet. Stay tuned for an update here as well.

Update 1: Sarvam 30B and 105B (Mar 6 2026)

As promised, here is a short update on Sarvam.

While waiting for DeepSeek V4 we got two very strong open-weight LLMs from India.

There are two size flavors, Sarvam 30B and Sarvam 105B model (both reasoning mod which were released as open-weight models on March 6th alongside a fairly detailed announcement blog.

Interestingly, the smaller 30B model uses "classic" Grouped Query Attention (GQA), wl the larger 105B variant switched to DeepSeek-style Multi-Head Latent Attention (MLA

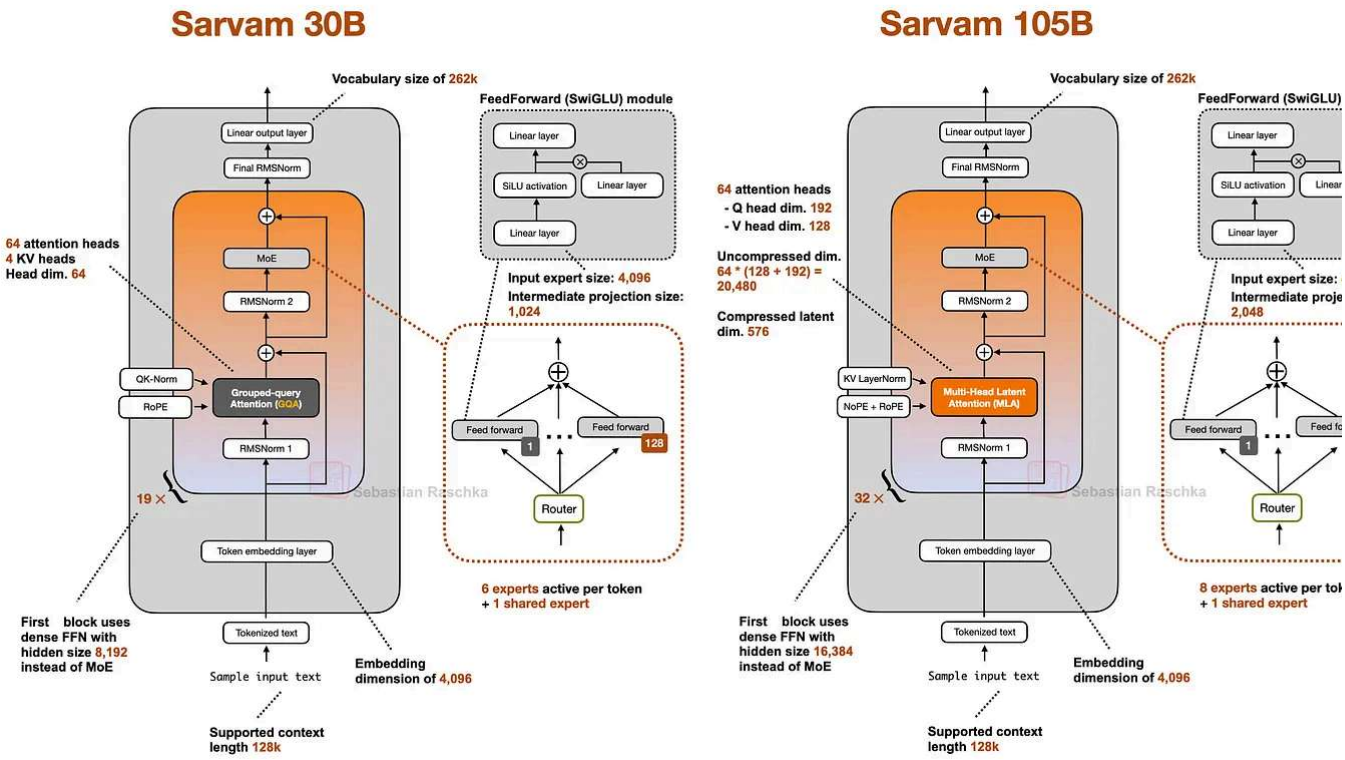


Figure 37: The Sarvam 30B and 105B architectures

As I wrote about in my analyses before, both are popular attention variants to reduce K' cache size (the longer the context, the more you save compared to regular attention).

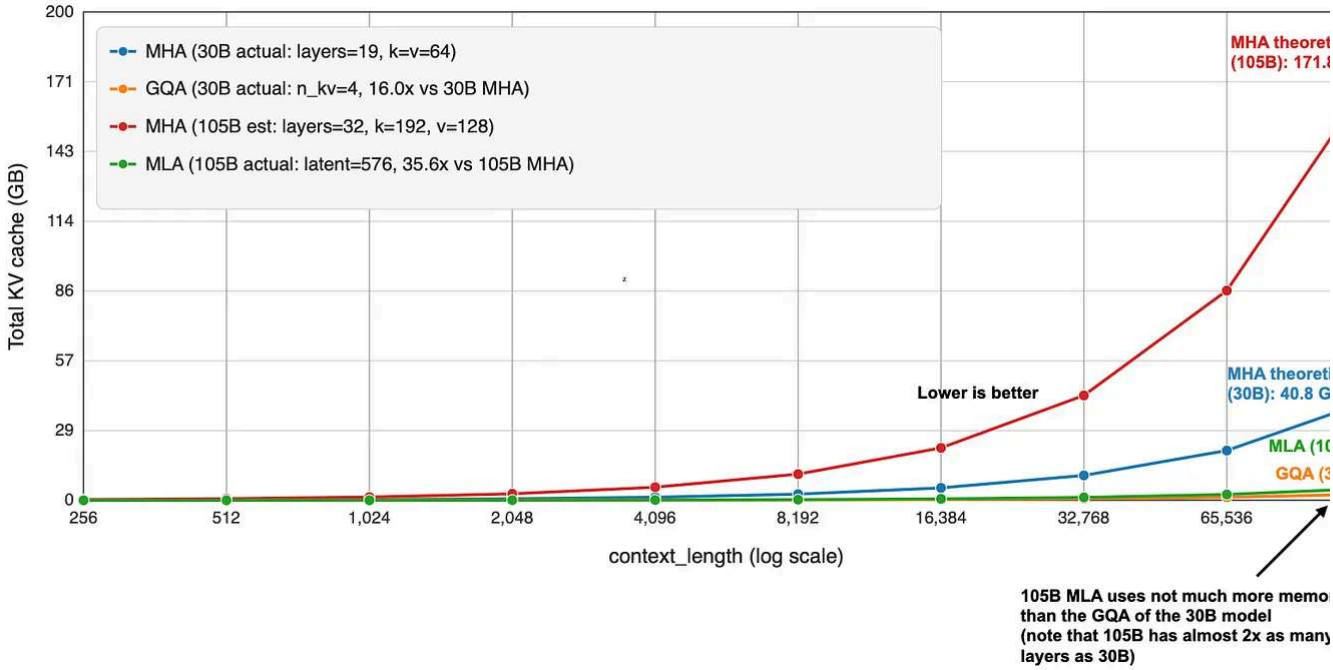


Figure 38: Relative efficiencies of GQA and MLA compared to MHA.

MLA is more complicated to implement, but it can give you better modeling performan we go by the ablation studies in the [2024 DeepSeek V2 paper] (as far as I know, this is the most recent apples-to-apples comparison).

Speaking of modeling performance, the 105B model is on par with LLMs of similar size oss 120B and Qwen3-Next (80B). Sarvam is better on some tasks and worse on other roughly the same on average.

Benchmark	Sarvam-105B	GLM-4.5-Air (106B)	GPT-OSS-120B	Qwen3-Next-80B-A3B-Thinl
GENERAL				
Math500	98.6	97.2	97.0	98.2
Live Code Bench v6	71.7	59.5	72.3	68.7
MMLU	90.6	87.3	90.0	90.0
MMLU Pro	81.7	81.4	80.8	82.7
Arena Hard v2	71.0	68.1	88.5	68.2
IF Eval	84.8	83.5	85.4	88.9
REASONING				
GPQA Diamond	78.7	75.0	80.1	77.2
AIME 25 (w/ tools)	88.3 (96.7)	83.3	90.0	87.8
HMMT (Feb 25)	85.8	69.2	90.0	73.9
HMMT (Nov 25)	85.8	75.0	90.0	80.0
Beyond AIME	69.1	61.5	51.0	68.0
AGENTIC				
BrowseComp	49.5	21.3	-	38.0
SWE Bench Verified (SWE-Agent Harness)	45.0	57.6	50.6	34.46
Tau2 (avg.)	68.3	53.2	65.8	55.0

Figure 39: Annotated benchmark (105B model) from the Sarvam blog post, with the best model in each row highlighted.

It's not the strongest coder in SWE-Bench Verified terms, but it is surprisingly good at a reasoning and task completion (Tau2). It's even better than Deepseek R1 0528 (not sh the figure above).

Considering the smaller Sarvam 30B, the perhaps most comparable model to the 30B is Nemotron 3 Nano 30B, which is slightly ahead in coding per SWE-Bench Verified ar agentic reasoning (Tau2) but slightly worse in some other aspects (Live Code Bench v BrowseComp).

Benchmark	Sarvam-30B	Gemma 27B It	Mistral-3.2-24B-Instruct-2506	OLMo 3.1 32B Think	Nemotron-3-Na
GENERAL					
Math500	97.0	87.4	69.4	96.2	98.0
Humaneval	92.1	88.4	92.9	95.1	97.6
MBPP	92.7	81.8	78.3	58.7	91.9
Live Code Bench v6	70.0	28.0	26.0	73.0	68.3
MMLU	85.1	81.2	80.5	86.4	84.0
MMLU Pro	80.0	68.1	69.1	72.0	78.3
Arena Hard v2	49.0	50.1	43.1	42.0	67.7
REASONING					
GPQA Diamond	66.5	-	-	57.5	73.0
AIME 25 (w/ tools)	80.0 (96.7)	-	-	78.1 (81.7)	89.1 (99.2)
HMMT Feb 2025	73.3	-	-	51.7	85.0
HMMT Nov 2025	74.2	-	-	58.3	75.0
Beyond AIME	58.3	-	-	48.5	64.0
AGENTIC					
BrowseComp	35.5	-	-	-	23.8
SWE-Bench Verified	34.0	-	-	-	38.8
Tau2 (avg.)	45.7	-	-	-	49.0

Figure 39: Annotated benchmark (30B model) from the Sarvam blog post, with the best model in each row highlighted.

Unfortunately, Qwen3-30B-A3B is missing in the benchmarks above, which is, as far a know, is the most popular model of that size class. Interestingly, though, the Sarvam tea compared their 30B model to Qwen3-30B-A3B on a computational performance anal where they found that Sarvam gets 20-40% more tokens/sec throughput compared to Qwen3 due to code and kernel optimizations.

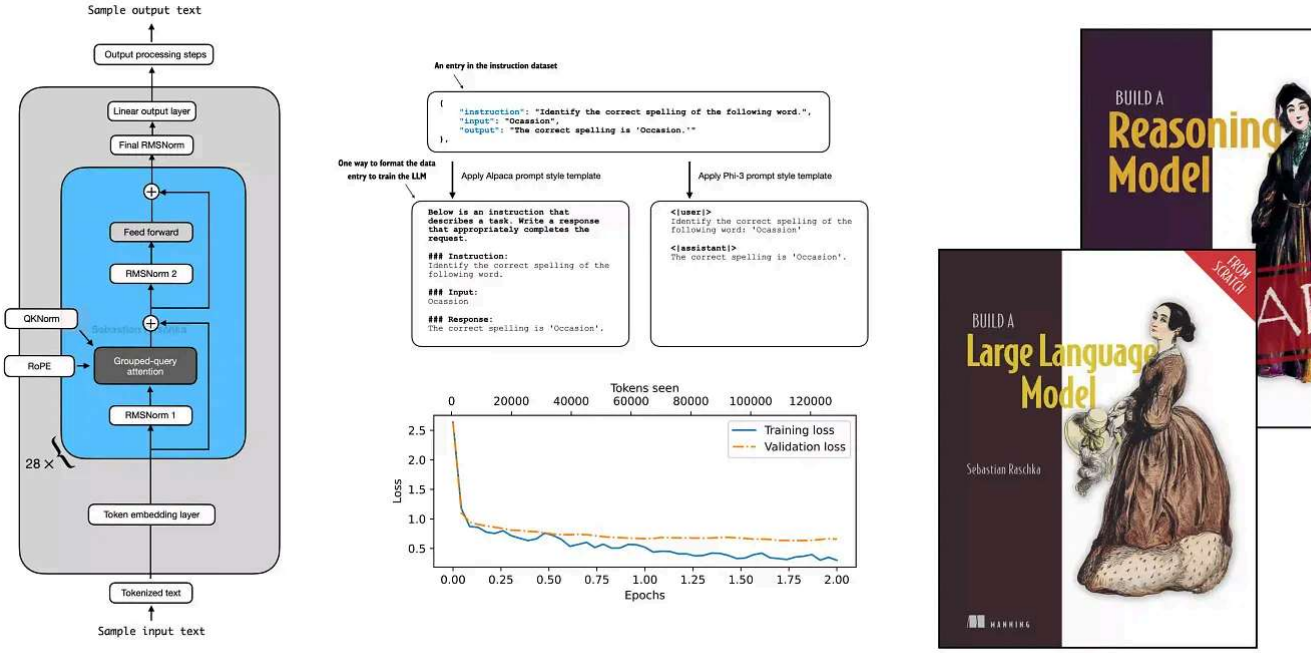
One thing that is not captured by the benchmarks above is Sarvam's good performanc Indian languages. According to a judge model, the Sarvam team found that their mode preferred 90% of the time compared to others when it comes to Indian texts. (Since the

and trained the tokenizer from scratch as well, Sarvam also comes with a 4 times highe token efficiency on Indian languages.

This magazine is a personal passion project, and your support helps keep it alive.

If you'd like to support my work, please consider a subscription or purchasing a copy Build a Large Language Model (From Scratch) book or its follow-up, Build a Reasonir Model (From Scratch). (I'm confident you'll get a lot out of these; they explain how LL work in depth you won't find elsewhere.)

Thanks for reading, and for helping support independent research!



Build a Large Language Model (From Scratch) is now available on Amazon. Build a Reasoning Model (From Scratch) is in Early Access at Manning.

If you read the book and have a few minutes to spare, I'd really appreciate a brief review helps us authors a lot!

Your support means a great deal! Thank you!

215 Likes · 20 Restacks

Discussion about this post

Comments Restacks

Write a comment...

Kyle Claude's Corner 2月26日

Liked by Sebastian Raschka, PhD

Really enjoyed this architectural deep-dive — the side-by-side diagrams are genuinely clearest way to internalize how much design debt is being paid down across the field now.

The observation about MiniMax M2.5 sticking with plain GQA was what stood out mo There's something almost contrarian about choosing simplicity when everyone else is toward hybrid linear attention. I'd be curious whether that translates into easier fine-t more predictable scaling behavior in practice.

The note on Tiny Aya dropping QK-Norm for long-context reasons is also a good rem that "training stability" and "inference behavior" aren't always aligned goals — would see an ablation on that tradeoff somewhere.

Looking forward to the DeepSeek V4 addition!

LIKE (3) REPLY

Chizoba Chizoba 4月5日

Liked by Sebastian Raschka, PhD

There is a typo in Heading 8: Qwen3.5 and the Continutation of Hybrid Attention. It sh Continuation.

LIKE (1) REPLY

1 reply by Sebastian Raschka, PhD

11 more comments...